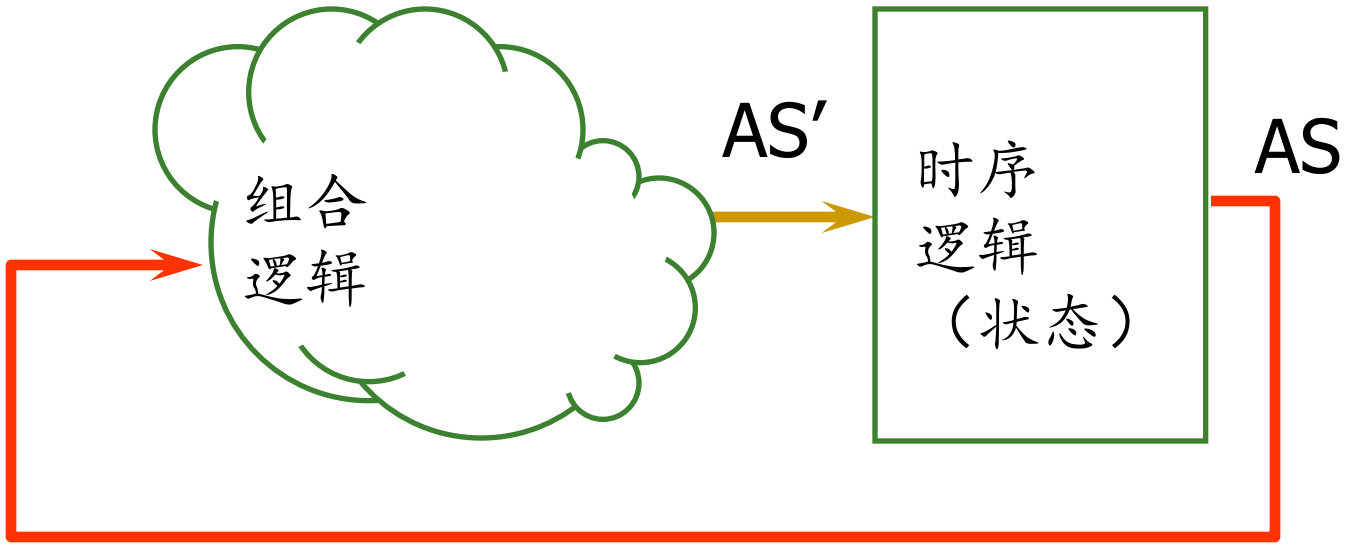
今天的内容

- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

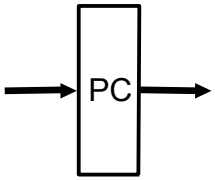
回顾

- 单周期机器



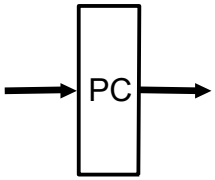
回顾：状态元素(State Elements)

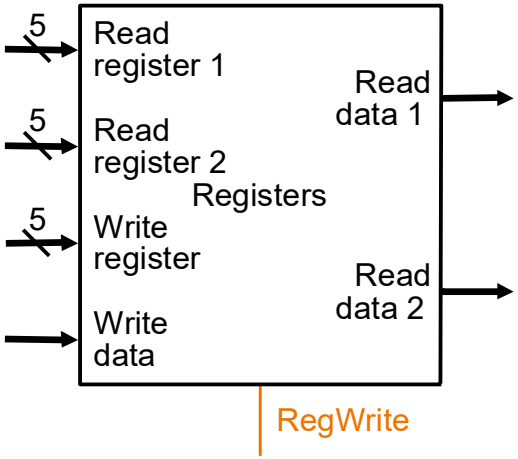
- 数据和控制输入



回顾：状态元素(State Elements)

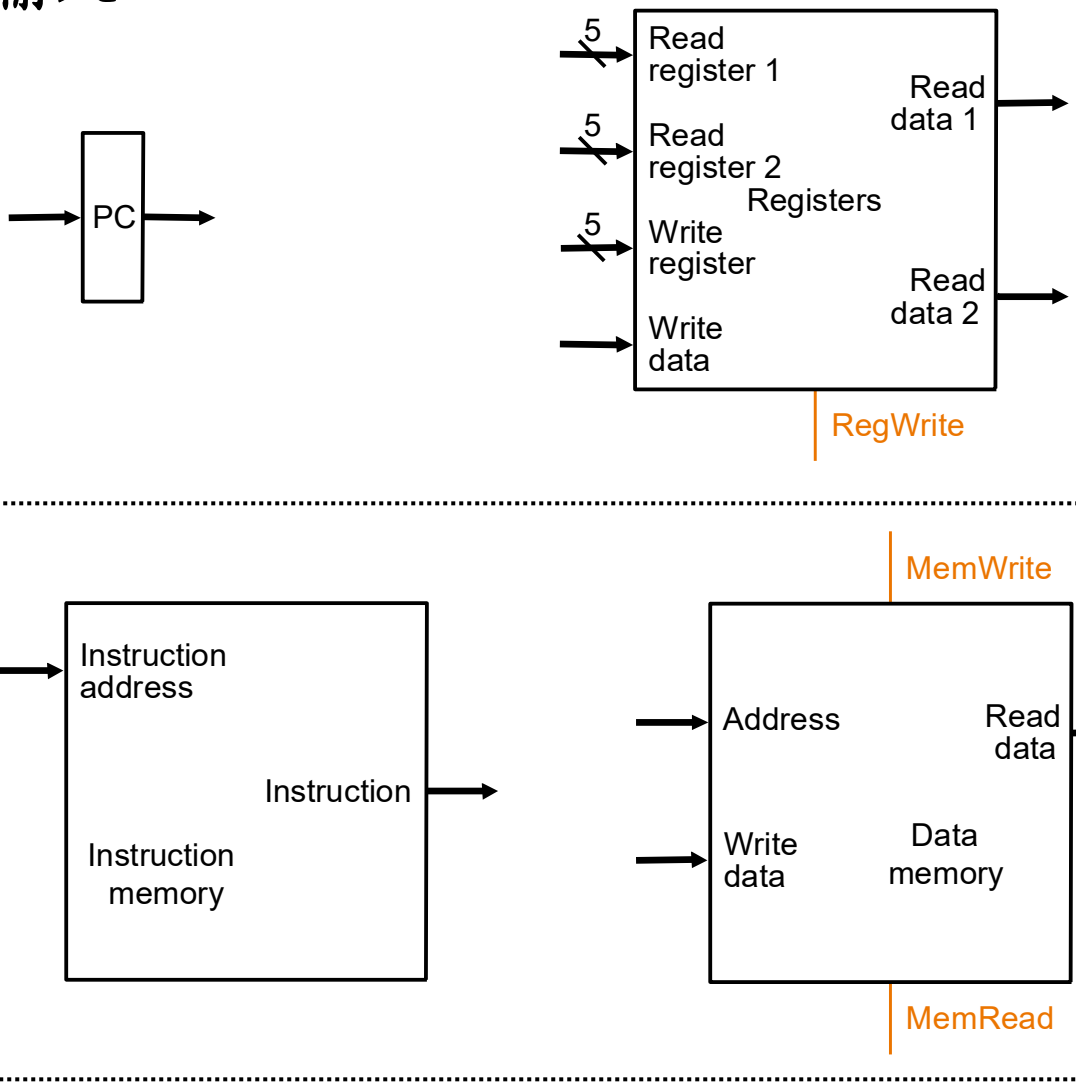
- 数据和控制输入





回顾：状态元素(State Elements)

- 数据和控制输入



回顾：我们假设：

- “魔法”内存和寄存器文件

- 组合读取 Combinational read
  - 读取数据端口的输出是寄存器文件内容和相应的读取选择端口的组合函数

回顾：我们假设：

- “魔法”内存和寄存器文件

- 组合读取 Combinational read
  - 读取数据端口的输出是寄存器文件内容和相应的读取选择端口的组合函数

- 同步写入 Synchronous write
  - 当写使能信号有效时，所选寄存器将在时钟上升沿更新。
  - 在时钟边缘之间不影响读取输出

回顾：我们假设：

- “魔法”内存和寄存器文件

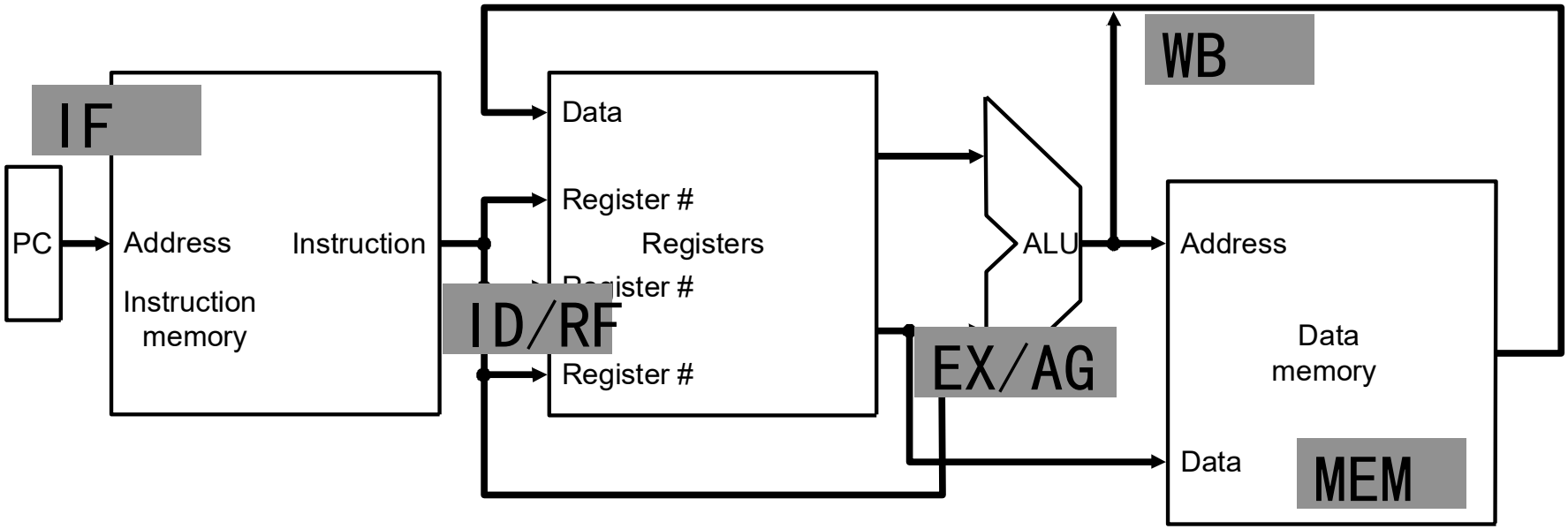
- 组合读取 Combinational read
  - 读取数据端口的输出是寄存器文件内容和相应的读取选择端口的组合函数

- 同步写入 Synchronous write
  - 当写使能信号有效时，所选寄存器将在时钟上升沿更新。
  - 在时钟边缘之间不影响读取输出

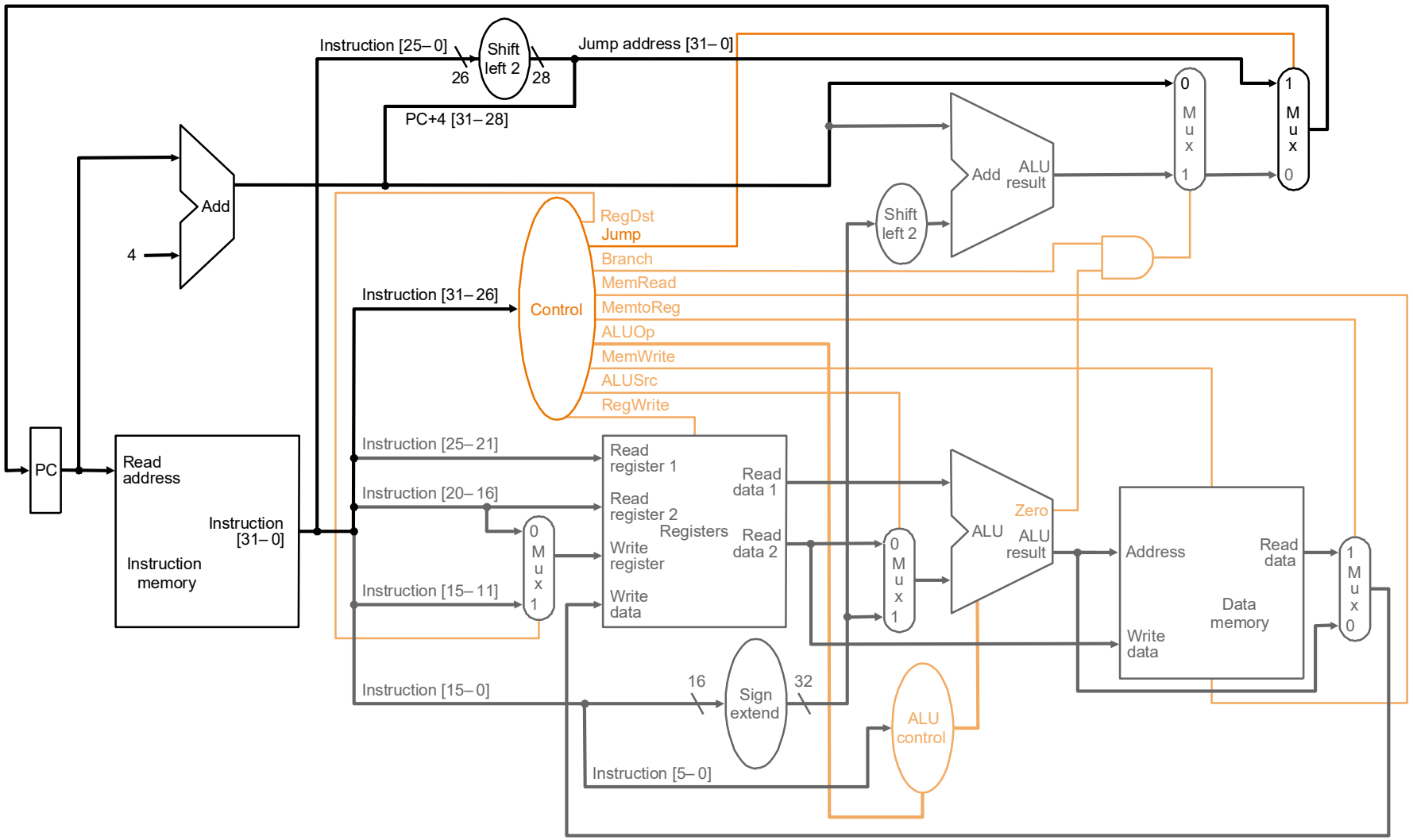
- 单周期同步内存 Single-cycle, synchronous memory

回顾：指令处理

- 5个通用步骤
  - 指令获取（IF）
  - 指令解码和寄存器操作数获取（ID/RF）
  - 执行/评估内存地址（EX/AG）
  - 内存操作数获取（MEM）
  - 存储/写回结果（WB）



回顾：完整MIPS数据通路



今天的内容

- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

R型 ALU 指令

- 汇编形式（如：寄存器间的有符号加法）

  ADD rd_{reg} rs_{reg} rt_{reg}

- 机器编码



- 指令语义

  if MEM[PC] == ADD rd rs rt

  GPR[rd] ← GPR[rs] + GPR[rt]

  PC ← PC + 4

ADD 示例①：读取操作数并计算

add $t0, $t1, $t2

执行前：PC=0x00400020。0x 开头表示十六进制，其余数值为十进制。

操作数	作用	执行前的值
$t0（rd）	目标寄存器，接收结果	0
$t1（rs）	第一个源寄存器	5
$t2（rt）	第二个源寄存器	7

① 读取：从 $t1 读出5，从 $t2 读出7；寄存器名指定数据存放的位置。

② 相加：5 + 7 = 12 → 写入 $t0

ALU 将两个源寄存器中的值相加，结果送到目标寄存器。

③ 更新 PC：0x00400020 + 4 = 0x00400024

一条指令占4字节，顺序执行时 PC 增加4。

$t0 的旧值0不参与加法，执行后被结果12覆盖。

ADD 示例②：数据路径与执行结果

add $t0, $t1, $t2

执行前：PC=0x00400020；$t0=0，$t1=5，$t2=7。

状态	执行前	执行后
$t0	0	12
$t1、$t2	5、7	5、7（不变）
PC	0x00400020	0x00400024
数据内存	原有内容	不变

① 读寄存器：rs=$t1、rt=$t2，两个读端口分别把5和7送入 ALU。

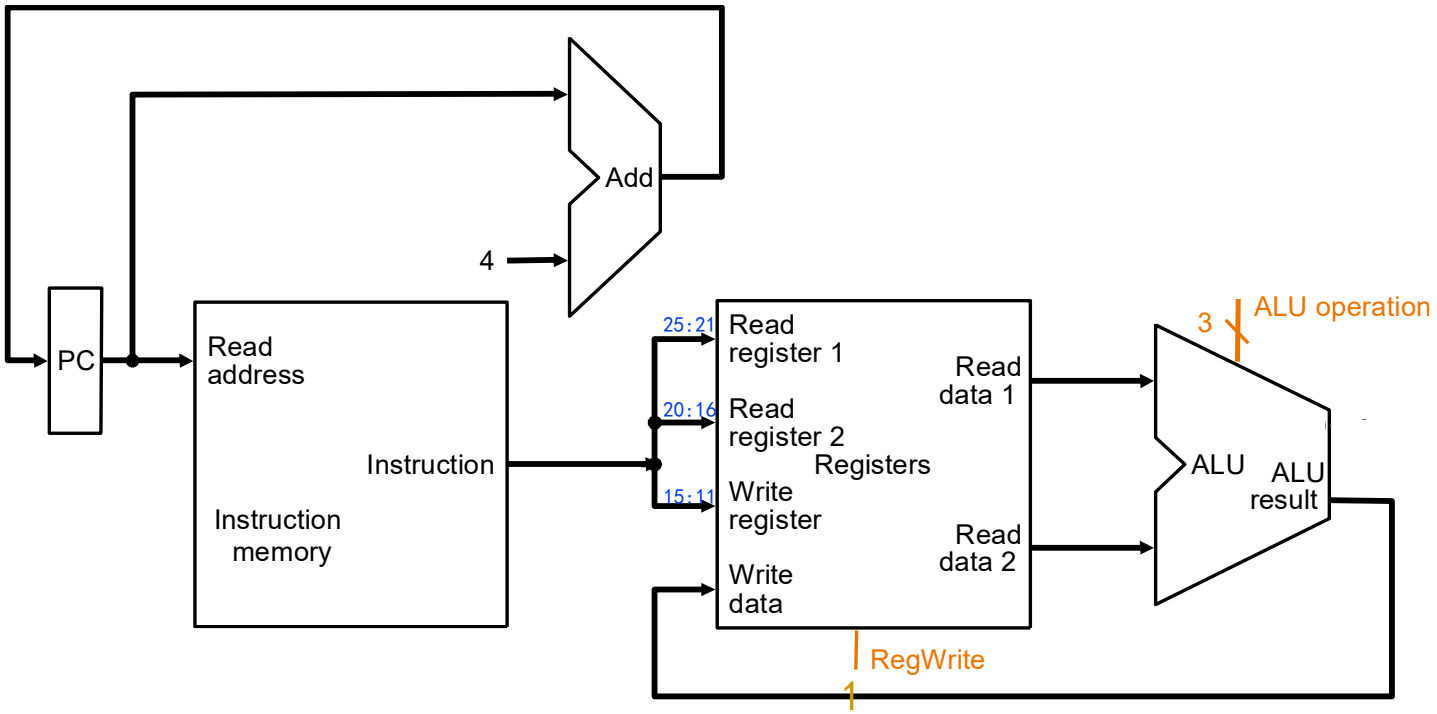
② ALU 运算：5 + 7 = 12

选择加法，ALU 输出12；这条指令不访问数据内存。

③ 写回：选择 rd=$t0 接收12；PC 同时更新为 PC+4，即0x00400024。

读取、运算和写回在一个时钟周期内完成，周期末更新 $t0 和 PC。

ALU 数据通路



add $t0, $t1, $t2
$t0 ← 5 + 7 = 12
PC: 0x00400020 → 0x00400024

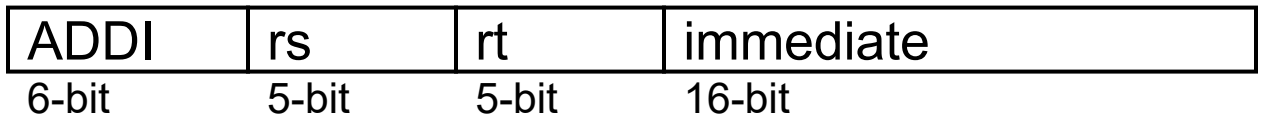
IF	ID	EX	MEM	WB

I型 ALU 指令

- 汇编形式（如：寄存器与立即数的有符号加法）

  ADDI rt_{reg} rs_{reg} $immediate_{16}$

- 机器编码



I-type

- 指令语义

  if MEM[PC] == ADDI rt rs immediate

  GPR[rt] ← GPR[rs] + sign-extend (immediate)

  PC ← PC + 4

ADDI 示例①：负立即数的符号扩展

addi $t0,$t1,-4

执行前：PC=0x00400020；$t0=0，$t1=12。指令中的立即数为−4。

位置与位宽	十六进制补码	有符号数值
指令内：16位	0xFFFC	−4
扩展后：32位	0xFFFFFFFC	−4（不变）

① 解释16位补码：0xFFFC = 65532；65532 − 2^{16} = −4

F=1111，符号位为1；无符号值减2^{16}，得到16位补码表示的数值。

② 扩展到32位：0xFFFC → 0xFFFFFFFC

把符号位1复制到新增的高16位，低16位保持原样。

③ 保持数值：0xFFFFFFFC 按32位补码仍为−4，可与32位寄存器相加。

若高位补0，0x0000FFFC 表示正数65532，就改变了原来的−4。

ADDI 示例②：相加并写回

addi $t0,$t1,-4

执行前：PC=0x00400020；$t0=0，$t1=12；−4已扩展为0xFFFFFFFC。

状态	执行前	执行后
$t0	0	8
$t1	12	12（不变）
PC	0x00400020	0x00400024
数据内存	原有内容	不变

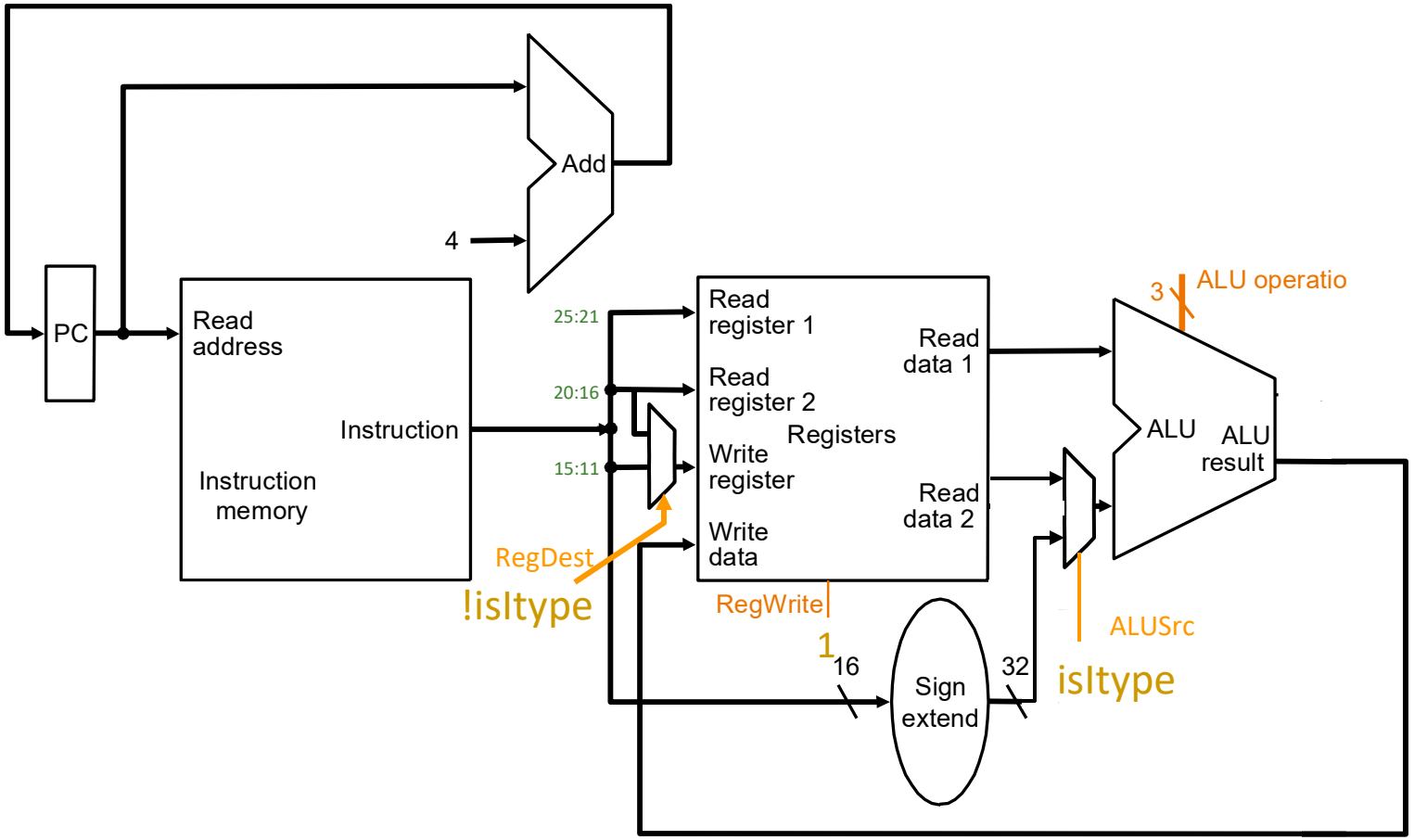
① 选择输入：ALU 输入A取 $t1 中的12；输入B取符号扩展后的立即数−4。

② 相加写回：12 + (−4) = 8 → 写入 rt=$t0
写回选择 ALU 输出，这条指令不读取数据内存。

③ 更新 PC：0x00400020 + 4 = 0x00400024
执行后从下一条顺序指令继续取指。

第二个加数来自立即数；目标寄存器 $t0 的旧值不参与运算。

R和I型ALU指令的数据通路



IF	ID	EX	MEM	WB

addi $t0, $t1, -4

$t0 ← 12 + (-4) = 8

PC: 0x00400020 → 0x00400024

今天的内容

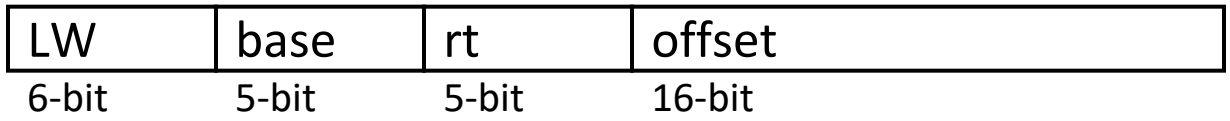
- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

加载指令（Load）

- 汇编形式（如：加载一个 4 字节的字）

  LW rt_{reg} $offset_{16}$ ($base_{reg}$)

- 机器编码



I-type

- 指令语义

if MEM[PC]==LW rt $offset_{16}$ (base)

EA = sign-extend(offset) + GPR[base]

GPR[rt] ← MEM[translate(EA)]

PC ← PC + 4

LW 示例①：用基址和偏移求地址

lw $t0,12($s0)

执行前：PC=0x00400020；$t0=0。内存按字节编址。

已知条件	数值	含义
基址寄存器 $s0	0x10010000	地址计算的起点
偏移量12	12字节 = 0xC字节	相对基址的距离
Mem32[0x1001000C]	42	该地址开始的32位字

① 解释操作数：12($s0) 表示基址加12字节；$t0 接收读出的数据。

② 扩展偏移量：0x000C（16位）→ 0x0000000C（32位）
12为正数，符号扩展时高16位补0。

③ ALU 计算地址：0x10010000 + 0x0000000C = 0x1001000C
这个结果送到数据内存的地址端，随后才能读取内容。

0x1001000C 是地址，该地址开始的32位字才是要读出的42。

LW 示例②：从内存读数并写回

lw $t0,12($s0)

执行前：PC=0x00400020；$s0=0x10010000，$t0=0；目标内存字为42。

状态	执行前	执行后
$t0	0	42
$s0	0x10010000	0x10010000（不变）
Mem32[0x1001000C]	42	42（不变）
PC	0x00400020	0x00400024

① 送入地址：0x10010000 + 12 = 0x1001000C

ALU 将计算出的地址送到数据内存的地址端。

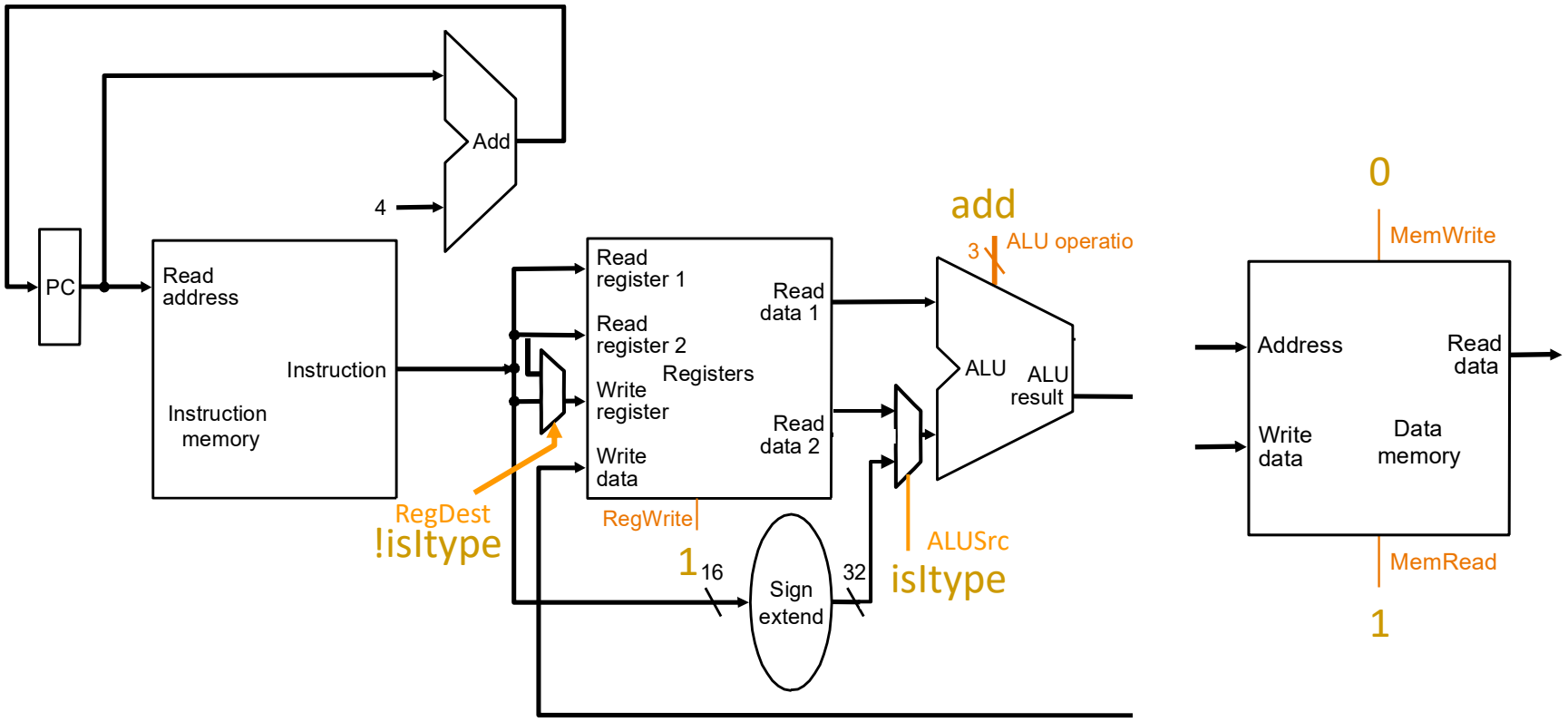
② 读取并写回：内存读出32位字42，选择该数据写回 rt=$t0。

③ 顺序执行：PC + 4 = 0x00400024

内存内容和基址 $s0 保持不变。

$t0 接收内存中的数据42，不接收地址0x1001000C。

LW 数据通路



lw $t0, 12($s0)
EA = 0x10010000 + 12 = 0x1001000C
$t0 ← Mem32[0x1001000C] = 42
PC: 0x00400020 → 0x00400024

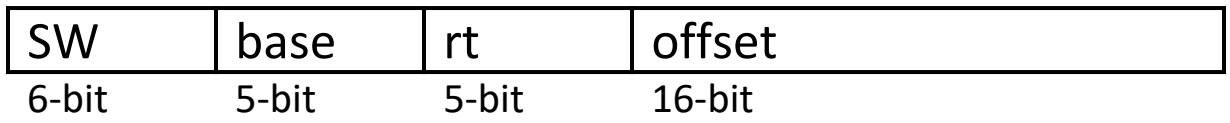
IF	ID	EX	MEM	WB

存储指令（Store）

- 汇编形式（如：存储一个 4 字节的字）

  SW rt_{reg} $offset_{16}$ ($base_{reg}$)

- 机器编码



I-type

- 指令语义

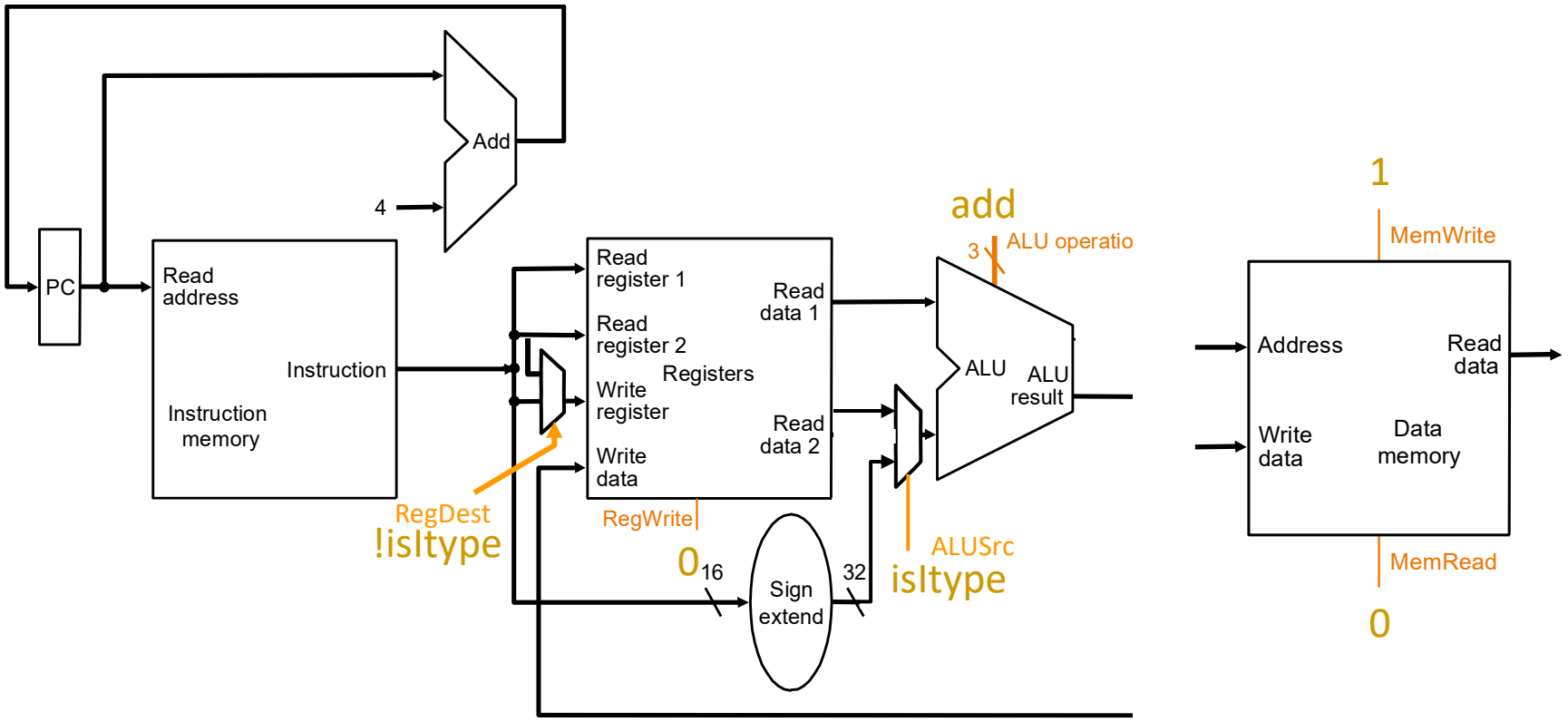
if MEM[PC]==SW rt $offset_{16}$ (base)

EA = sign-extend(offset) + GPR[base]

MEM[translate(EA)] ← GPR[rt]

PC ← PC + 4

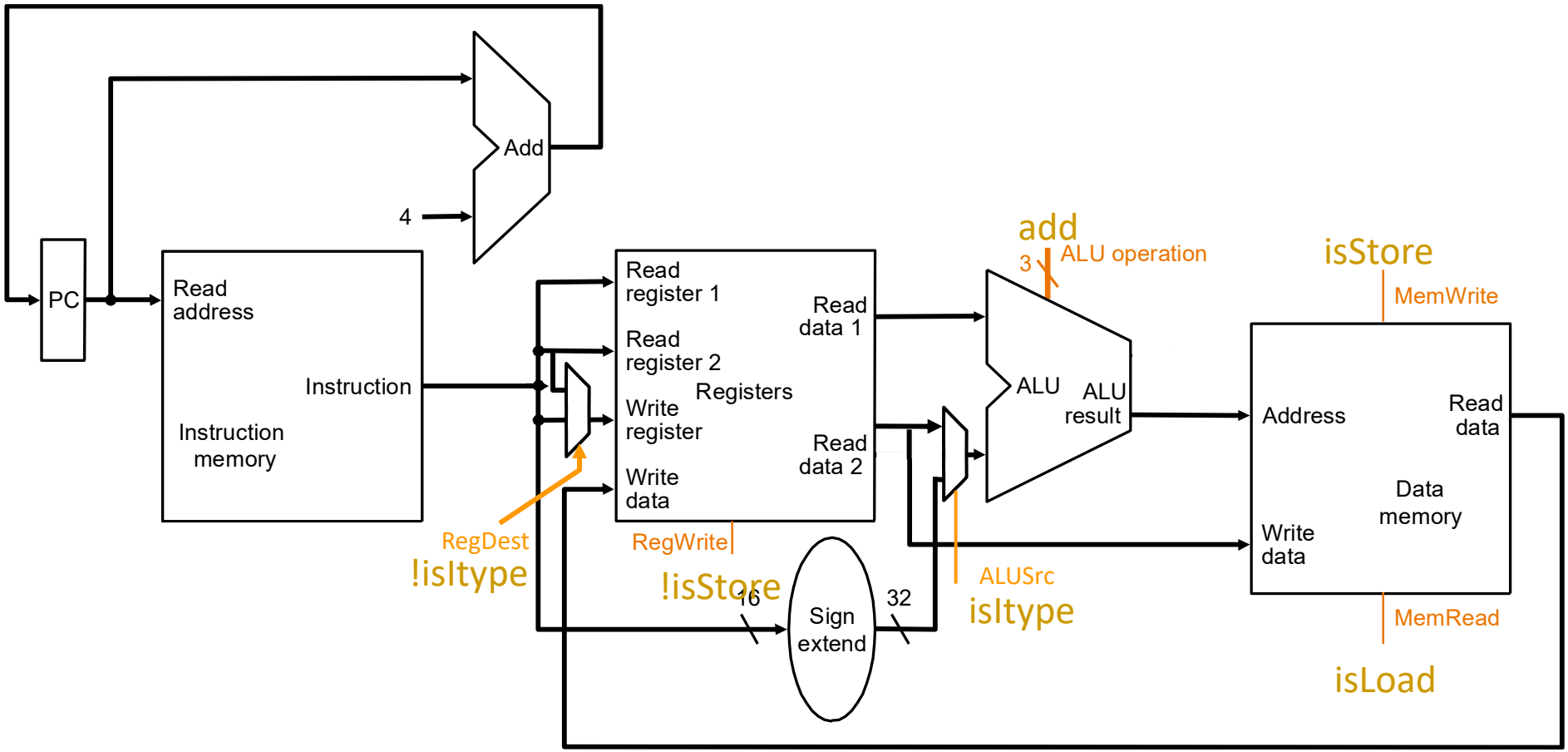
SW数据通路



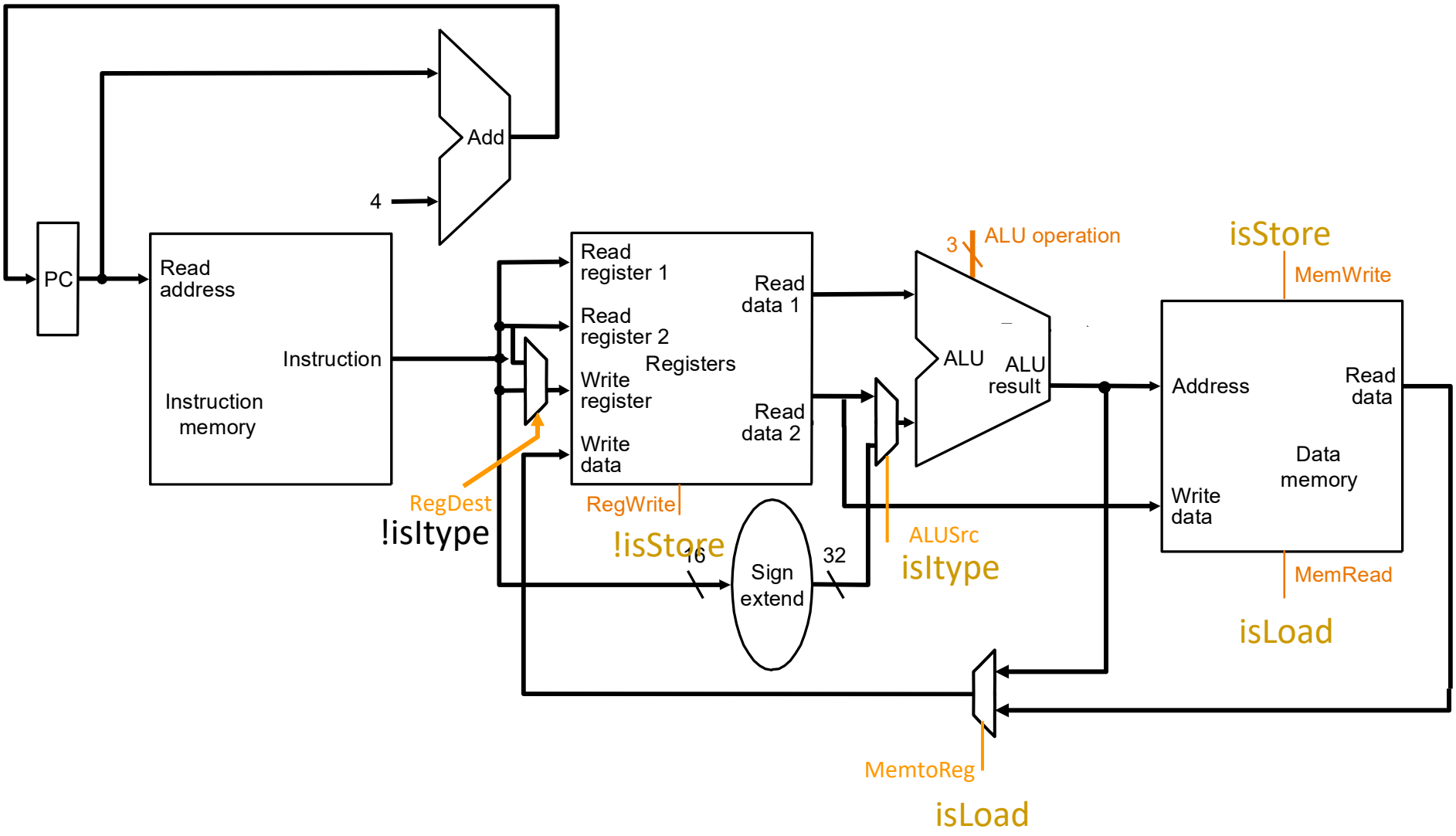
sw $t0, 16($s0)
EA = 0x10010000 + 16 = 0x10010010
Mem32[0x10010010] ← 42
PC: 0x00400020 → 0x00400024

IF	ID	EX	MEM	WB

加载/存储指令数据通路


Add
4
PC
Read address
Instruction
Instruction memory
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
RegDest
!isItype
RegWrite
!isStore
16
Sign extend
32
add
3
ALU operation
ALU
ALU result
ALUSrc
isItype
isStore
MemWrite
Address
Read data
Data memory
Write data
MemRead
isLoad

非控制流指令数据通路


Add
4
PC
Read address
Instruction
Instruction memory
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
RegDest
!isItype
RegWrite
!isStore
16
Sign extend
32
3
ALU operation
ALU
ALU result
ALUSrc
isItype
isStore
MemWrite
Address
Read data
Data memory
Write data
MemRead
isLoad
MemtoReg
isLoad

今天的内容

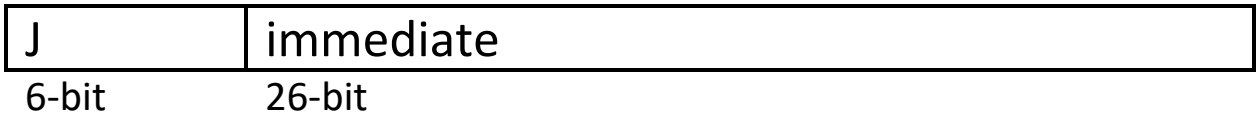
- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

无条件跳转指令

- 汇编形式

J target_address

- 机器编码



J-type

- 指令语义

if MEM[PC]==J immediate$_{26}$

target = { (PC+4)[31:28], immediate$_{26}$, 2'b00 }

PC ← target

J 示例①：目标地址与指令字段

j 0x00400040

执行前：PC=0x00400020；目标地址为0x00400040。采用无延迟槽模型。

名称	十六进制值	位宽与含义
汇编中的目标地址	0x00400040	32位：下一条取指地址
PC+4	0x00400024	32位：提供目标的高4位
指令的 target 字段	0x00100010	26位：目标地址的[27:2]

① 省去低2位：地址按4字节对齐，低2位为00，无需保存到字段中。

② 求字段值：0x00400040 ÷ 4 = 0x00100010

本例取目标地址的[27:2]，得到26位 target 字段。

③ 地址与字段：汇编写32位地址，指令的 target 字段只保存其中26位。

执行时需要补回低2位，并从 PC+4 取得目标地址的高4位。

J 示例②：拼接地址并更新 PC

j 0x00400040

执行前：PC=0x00400020；target=0x00100010（26位）。采用无延迟槽模型。

状态	执行前	执行后
PC（取指地址）	0x00400020	0x00400040
通用寄存器	原有数值	全部不变
数据内存	原有内容	全部不变

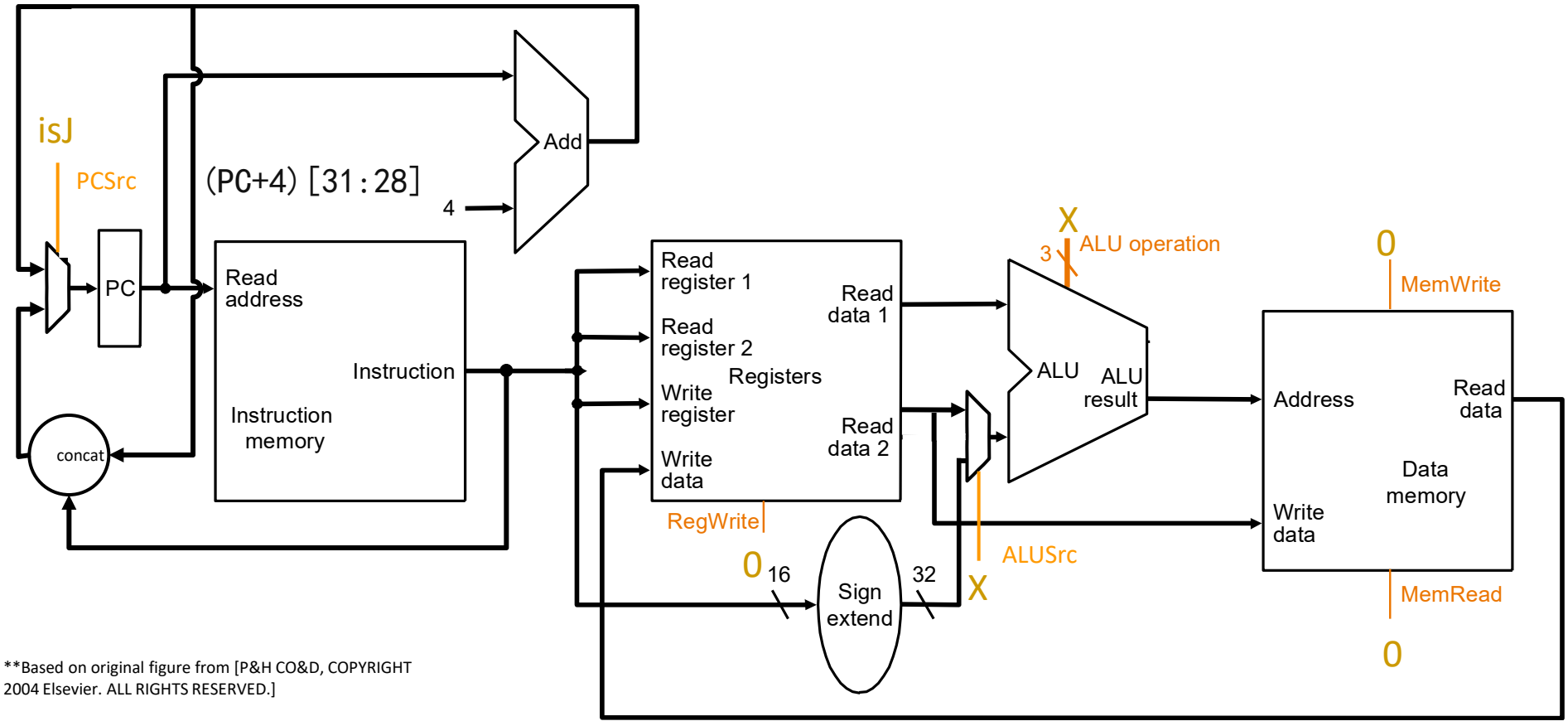
① 补回低2位：0x00100010 × 4 = 0x0400040（28位）
将26位 target 左移2位，在最低端补两个0。

② 取得高4位：PC+4 = 0x00400024；[31:28] = 0x0
取 PC+4 的高4位，放在目标地址的最高端。

③ 拼接并写入 PC：{0x0（4位），0x0400040（28位）} = 0x00400040
大括号表示位拼接；将所得32位地址写入 PC。

下一次直接从0x00400040取指，不先执行0x00400024处的指令。

无条件跳转指令数据通路



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

j 0x00400040

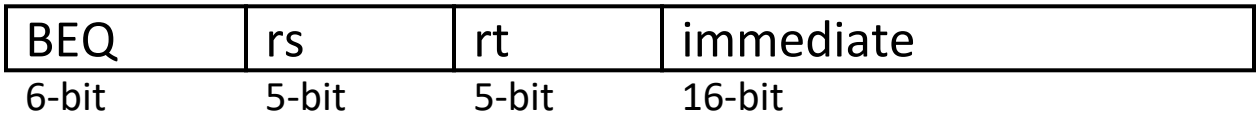
PC ← { (PC+4)[31:28], 0x00100010, 00 } = 0x00400040

条件分支指令

- 汇编形式（如：相等时分支）

  BEQ rs_{reg} rt_{reg} $immediate_{16}$

- 机器编码



I-type

- 指令语义（假设无分支延迟槽）

if MEM[PC]==BEQ rs rt $immediate_{16}$

target = PC + 4 + sign-extend(immediate) x 4

if GPR[rs]==GPR[rt] then PC ← target

else PC ← PC + 4

BEQ 示例①：分支偏移怎样计算

beq $t0, $t1, L

执行前：PC=0x00400020；$t0=5，$t1=5。

标号 L 的地址为0x00400030；采用无延迟槽模型。

计算对象	计算或数值	含义
基准地址 PC+4	0x00400024	下一条指令地址
目标与基准的差	0x00400030 − 0x00400024 = 0xC	12字节
16位立即数字段	12 ÷ 4 = 3，即0x0003	偏移3个指令字

① 比较的对象：beq 比较 $t0 和 $t1；相等时选择标号 L 的地址。

② 换算为字节：3 × 4 = 12字节 = 0xC字节

立即数3先符号扩展，再左移2位；每个指令字占4字节。

③ 计算候选目标：PC+4 + 3×4 = 0x00400024 + 0xC = 0x00400030

分支加法器把基准地址和字节偏移相加。

偏移从 PC+4 起算；字段3的单位是指令字，换成字节后才能相加。

BEQ 示例②：相等时跳转

beq $t0, $t1, L

执行前：PC=0x00400020；$t0=5，$t1=5；立即数为3。

标号 L 的地址为0x00400030；采用无延迟槽模型。

状态	执行前	执行后
$t0、$t1	5、5	5、5（不变）
PC（取指地址）	0x00400020	0x00400030
数据内存	原有内容	不变

① 比较寄存器：5 − 5 = 0 → Zero=1

两数相等；对于 BEQ，bcond=Zero=1。

② 并行计算目标：PC+4 + 3×4 = 0x00400024 + 0xC = 0x00400030

分支加法器计算目标地址，与寄存器比较并行进行。

③ 选择下一 PC：BEQ 条件成立，PCSrc2=1，选择0x00400030写入 PC。

下一次直接从 L 取指；比较不改写寄存器。

BEQ 示例③：不相等时顺序执行

beq $t0, $t1, L

执行前：PC=0x00400020；$t0=5，$t1=7；立即数为3。

标号 L 的地址为0x00400030；采用无延迟槽模型。

状态	执行前	执行后
$t0、$t1	5、7	5、7（不变）
PC（取指地址）	0x00400020	0x00400024
数据内存	原有内容	不变

① 比较寄存器：5 − 7 = −2 ≠ 0 → Zero=0

两数不相等；对于 BEQ，bcond=Zero=0。

② 仍计算候选目标：PC+4 + 3×4 = 0x00400030

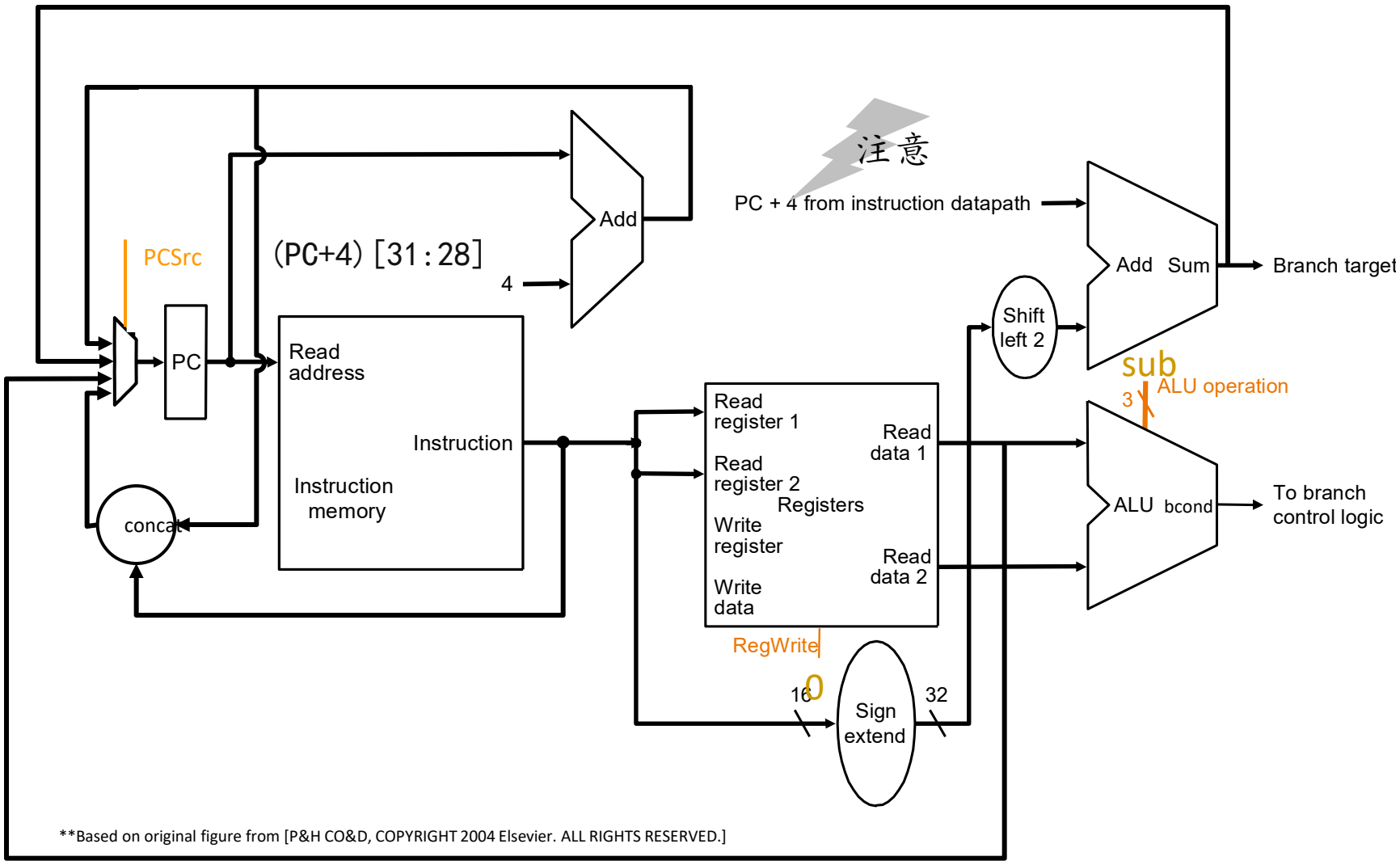
目标加法器仍输出这个地址，但分支条件不成立。

③ 选择下一 PC：PC+4 = 0x00400024

PCSrc2=0，选择顺序地址，寄存器和数据内存保持不变。

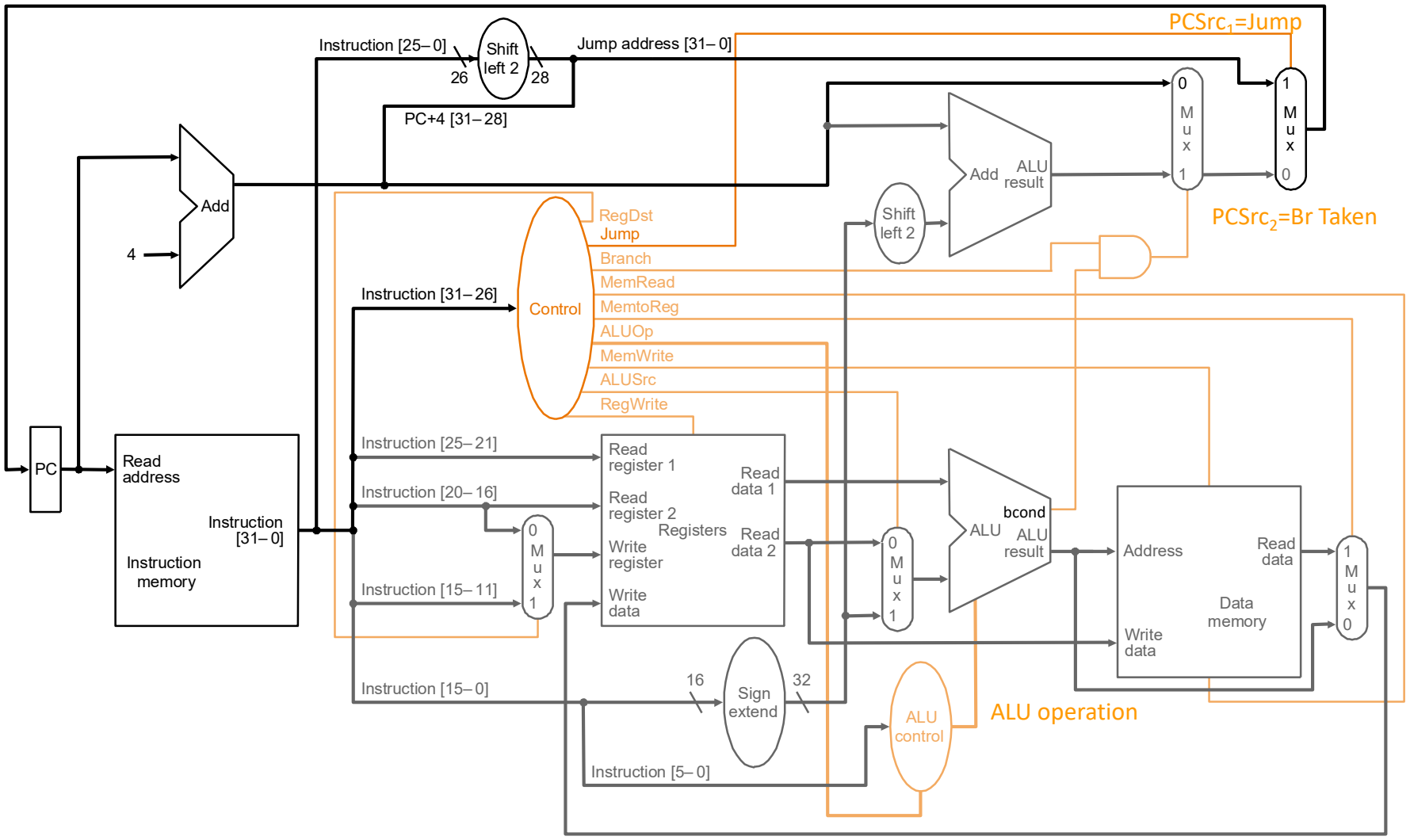
候选目标仍是0x00400030；比较结果决定是否采用它。

条件分支指令数据路径



**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

放在一起

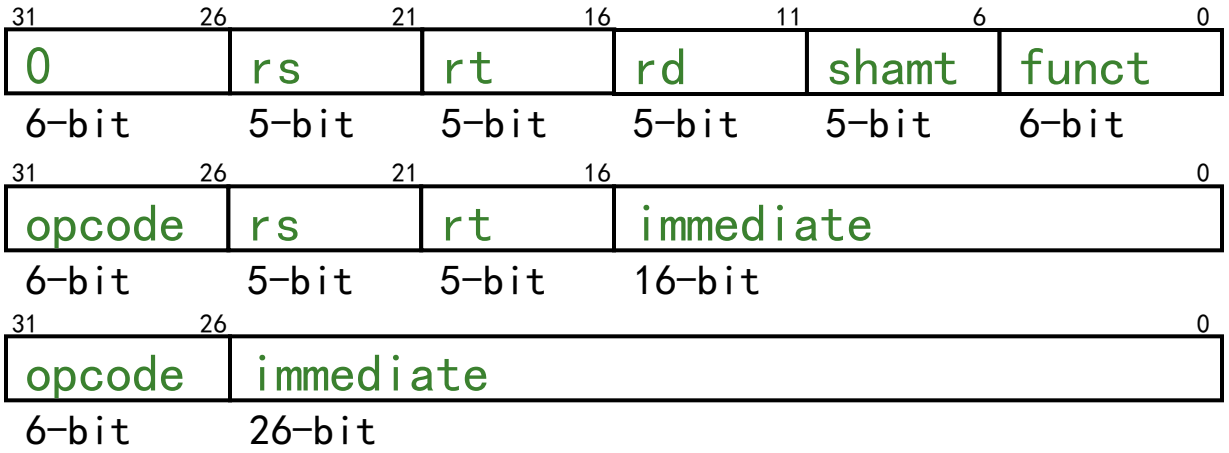

PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
0
M
u
x
1
1
M
u
x
0
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
ALU control
ALU operation
Address
Read data
Data memory
Write data

今天的内容

- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

单周期硬连线控制

- 作为 Inst=MEM[PC] 的组合函数



- 本课数据通路的范围
  - R型ALU、ADDI；LW、SW；BEQ；J
  - R型ALU：ADD/SUB/AND/OR/XOR/NOR
  - 扩展：其他I型、BNE/BLEZ/BGTZ
  - JR/JAL/JALR 需另加译码或通路

单比特控制信号

	当去断言（de-asserted）	当断言（asserted）	Equation
RegDest	GPR write select according to rt, i.e., inst[20:16]	GPR write select according to rd, i.e., inst[15:11]	opcode==0
ALUSrc	2nd ALU input from 2nd GPR read port	2nd ALU input from sign-extended 16-bit immediate	(opcode!=0) && (opcode!=BEQ) && (opcode!=BNE)
MemtoReg	Steer ALU result to GPR write port	steer memory load to GPR wr. port	opcode==LW
RegWrite	GPR write disabled	GPR write enabled	(opcode!=SW) && (opcode!=Bxx) && (opcode!=J) && (opcode!=JR))

单比特控制信号

	When De-asserted	When asserted	Equation
MemRead	Memory read disabled	Memory read port return load value	opcode==LW
MemWrite	Memory write disabled	Memory write enabled	opcode==SW
$PCSrc_1$	According to $PCSrc_2$	next PC is based on 26-bit immediate jump target	(opcode==J) || (opcode==JAL)
$PCSrc_2$	next PC = PC + 4	next PC is based on 16-bit immediate branch target	(opcode==Bxx) && "bcond is satisfied"

ALU 控制

- **opcode**

  ‘0’ ⇒ select operation according to funct

  ‘ALUi’ ⇒ selection operation according to opcode

  ‘LW’ ⇒ select addition

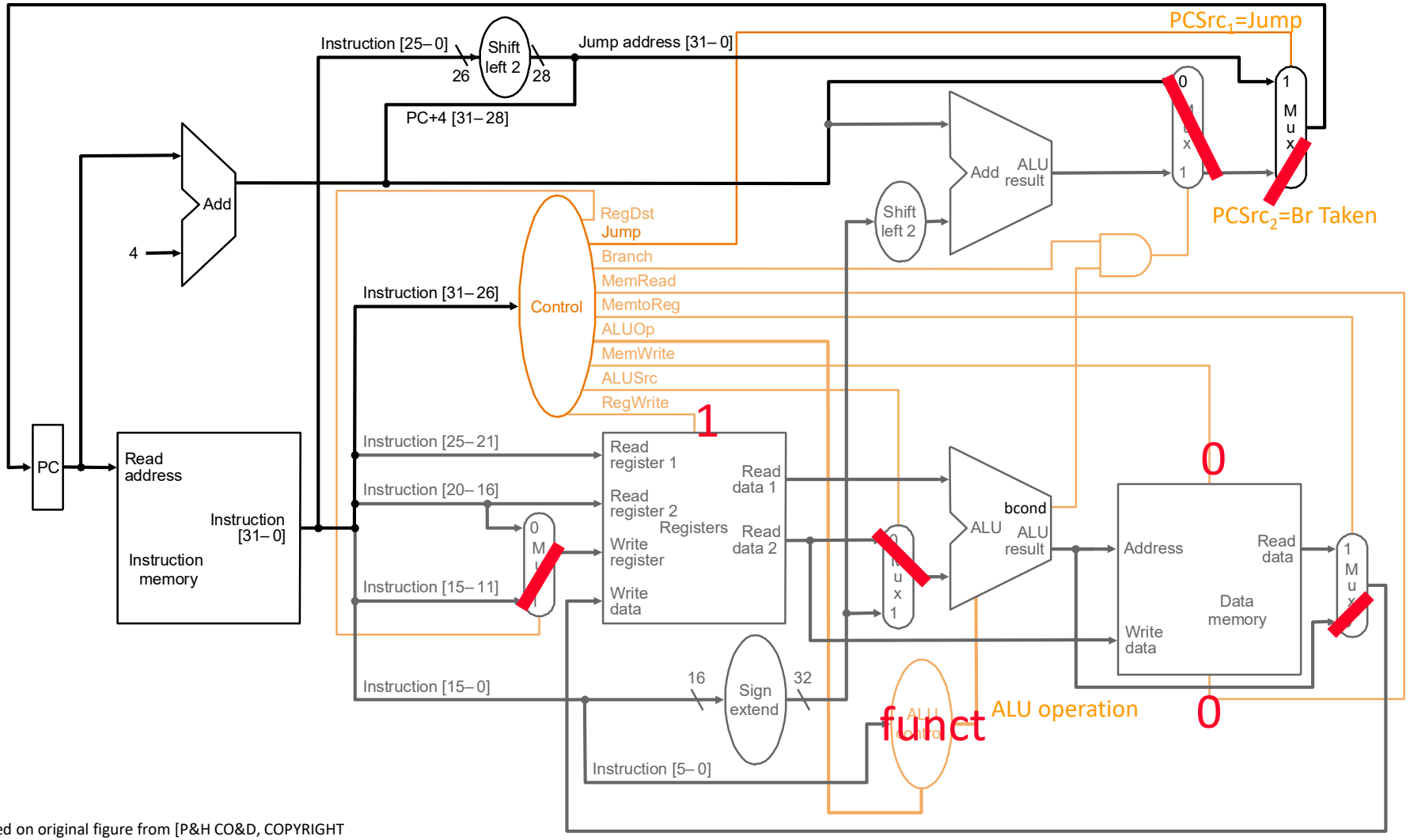
  ‘SW’ ⇒ select addition

  ‘Bxx’ ⇒ select bcond generation function

  __ ⇒ don’t care

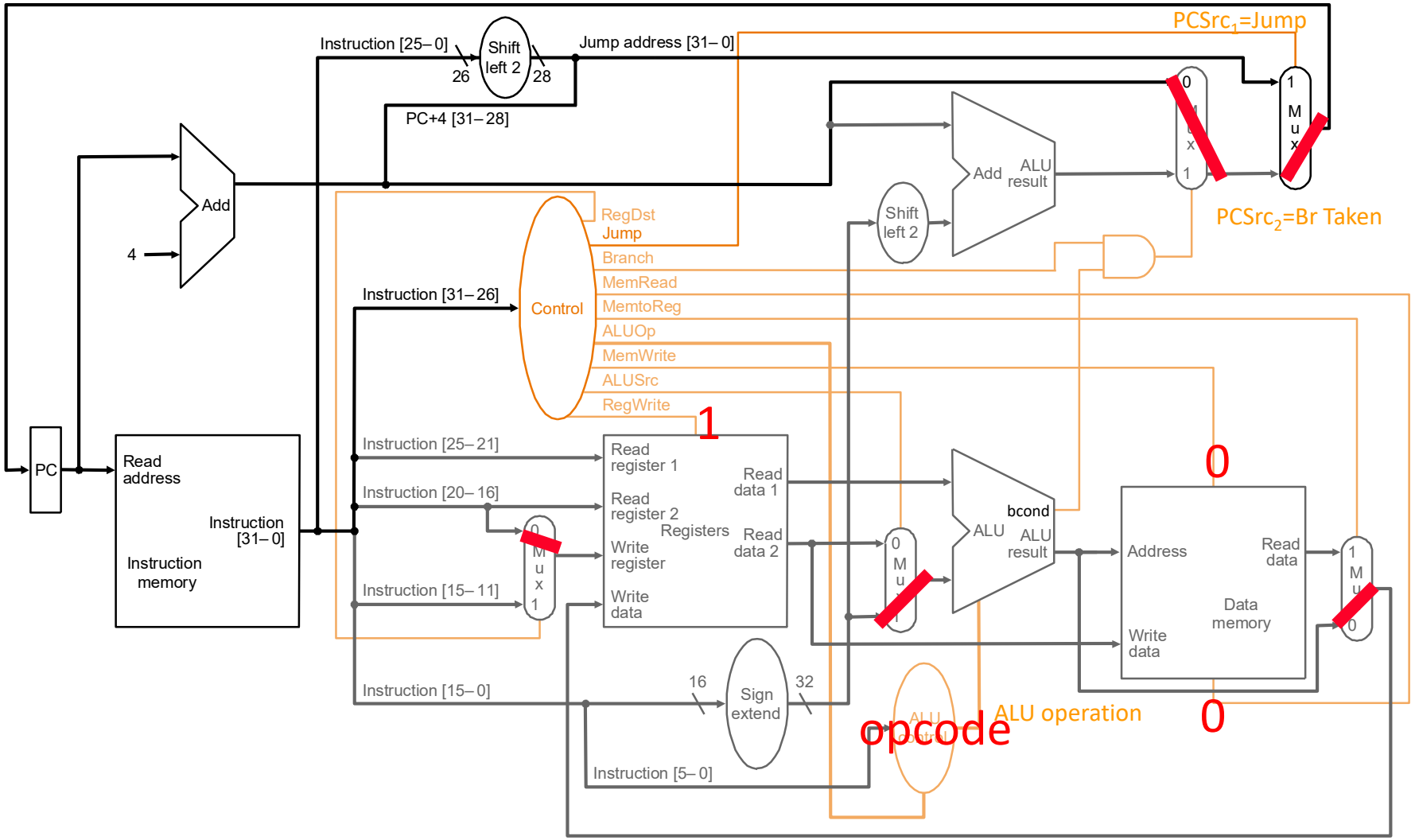
- **ALU 操作例子**
  - ADD, SUB, AND, OR, XOR, NOR, etc.
  - bcond on equal, not equal, LE zero, GT zero, etc.

R型 ALU 指令

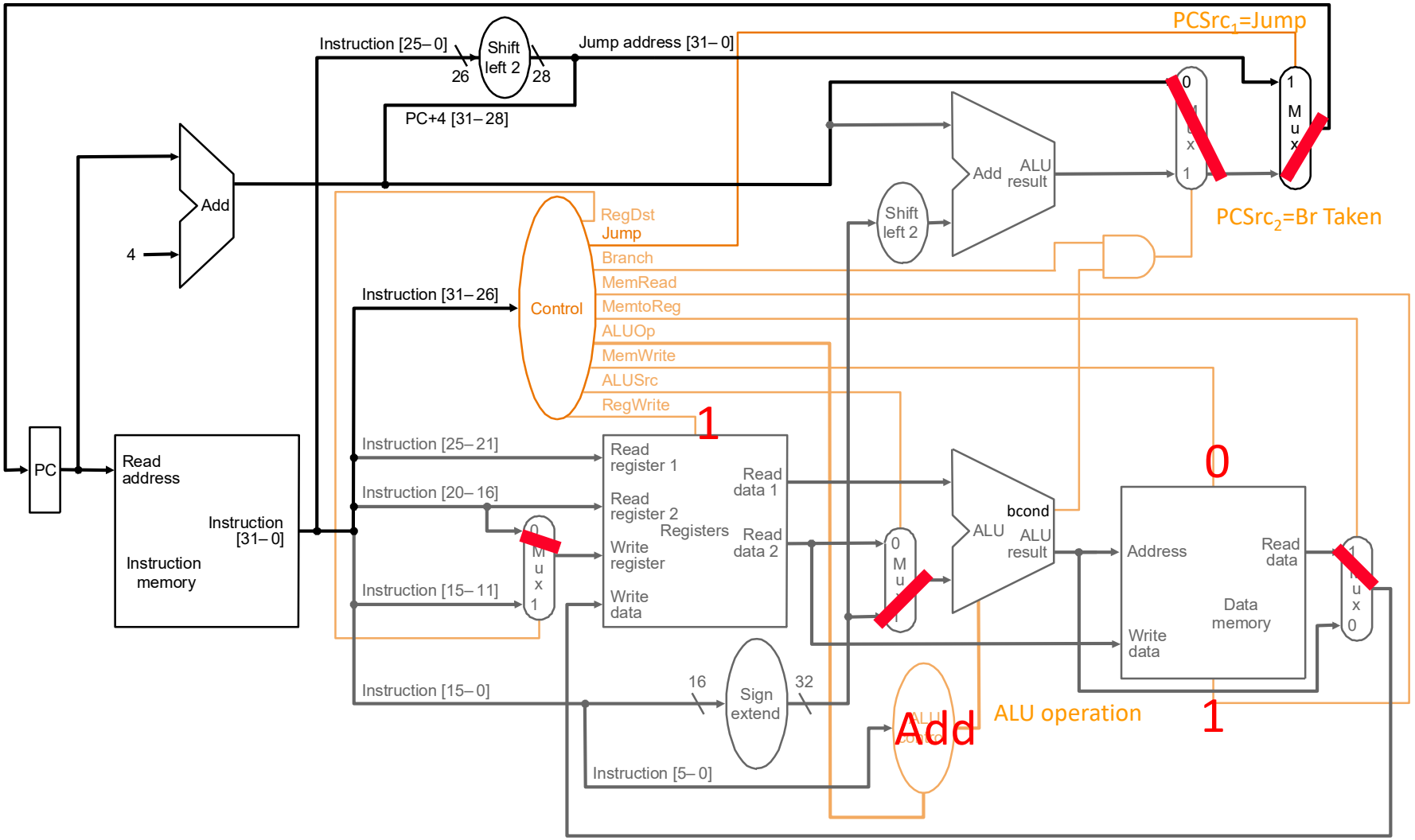


**Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

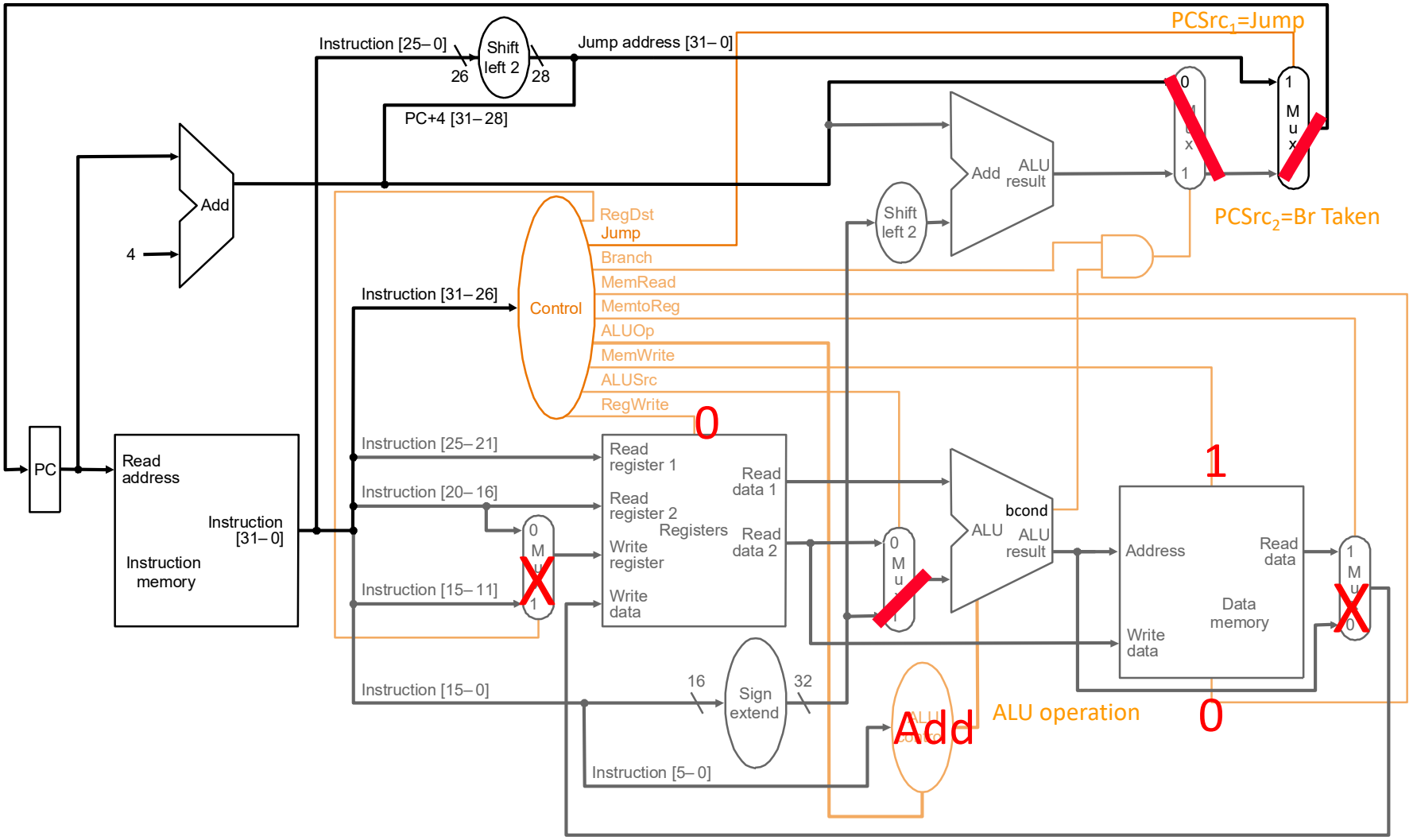
I型 ALU 指令


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
0
ALU operation
opcode
Address
Read data
Data memory
Write data
0
M u x
1

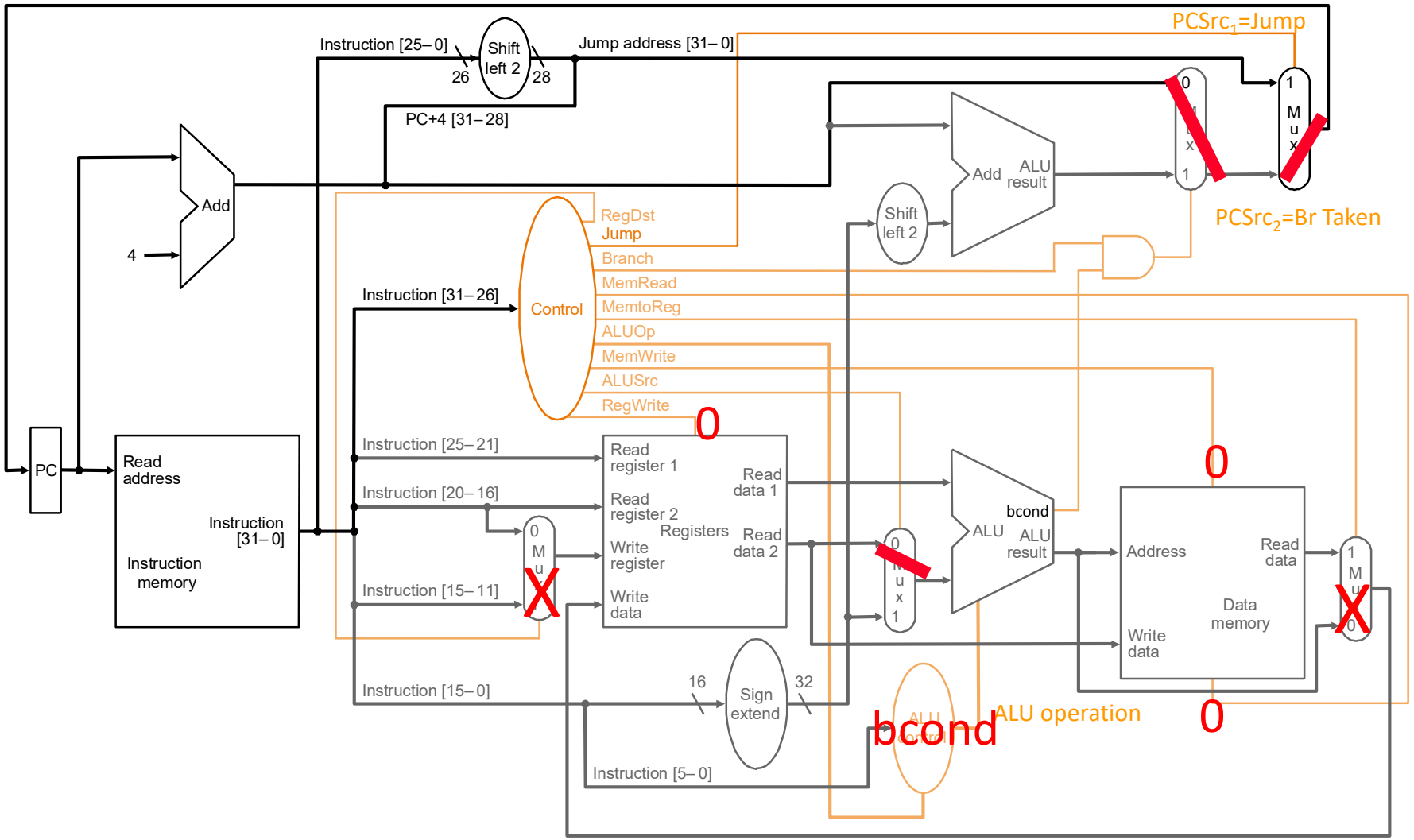
LW


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
Add
ALU result
Shift left 2
0
M u x
1
1
M u x
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
1
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
0
M u x
1
16
Sign extend
32
ALU
bcond
ALU result
Add
ALU operation
0
Address
Read data
Data memory
Write data
1
M u x
0
1

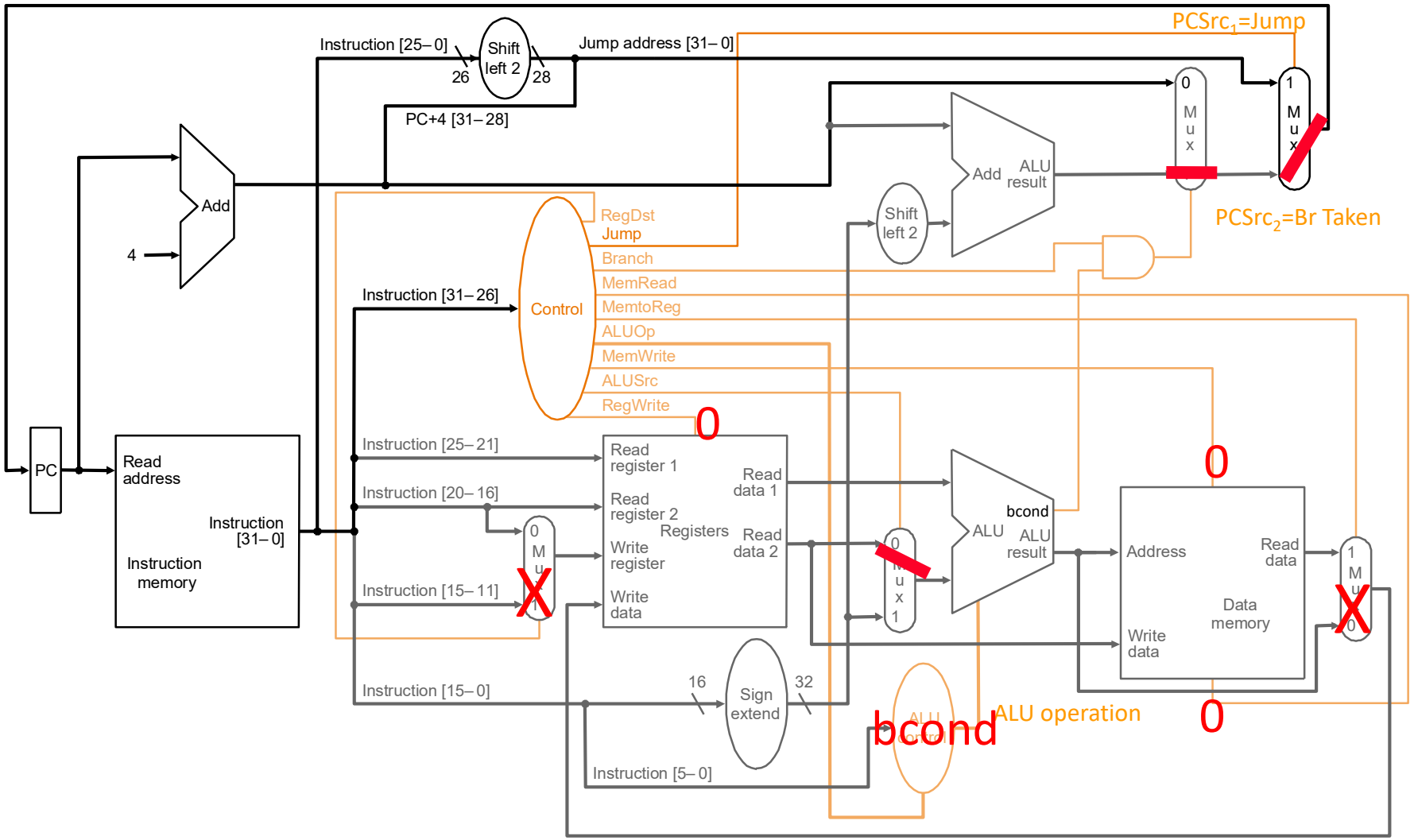
SW


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
Add
ALU result
Shift left 2
0
M u x
1
1
M u x
PCSrc2=Br Taken
Instruction [31–26]
Control
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
0
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
0
M u x
1
X
16
Sign extend
32
ALU
bcond
ALU result
Add
ALU operation
1
0
Address
Read data
Data memory
Write data
1
M u x
0
X

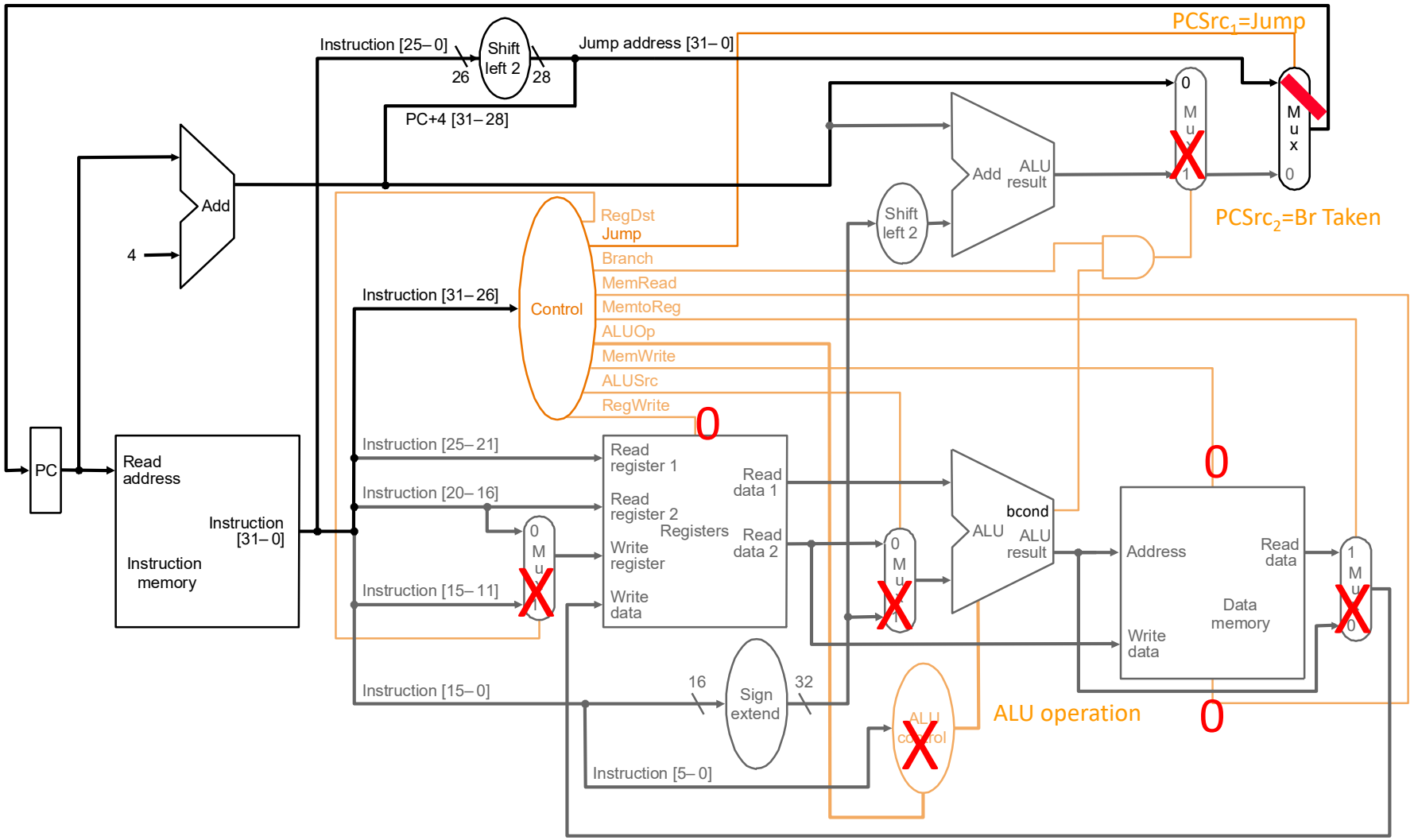
条件分支（不跳转）


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
0
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
ALU operation
bcond
Address
Read data
Data memory
Write data
0
0
0

条件分支（跳转）


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
0
M u x
1
M u x
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
0
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
0
M u x
1
X
16
Sign extend
32
ALU
bcond
ALU result
ALU operation
bcond
0
Address
Read data
Data memory
Write data
0
1
M u x
0
X
0

跳转


PCSrc1=Jump
Instruction [25–0]
Shift left 2
26
28
Jump address [31–0]
PC+4 [31–28]
Add
4
M u x
0
1
Add
ALU result
Shift left 2
PCSrc2=Br Taken
RegDst
Jump
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
0
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
ALU
bcond
ALU result
ALU control
ALU operation
Address
Read data
Data memory
Write data
0
0
0
1

控制盒中是什么？

- **组合逻辑 → 硬件控制**
  - 思路：基于指令生成的控制信号是组合逻辑的结果。
  - 在单周期微架构中是必需的…

控制盒中是什么？

- **组合逻辑 → 硬件控制**
  - 思路：基于指令生成的控制信号是组合逻辑的结果。
  - 在单周期微架构中是必需的…

- **时序逻辑 → 时序/微程序控制**
  - 思路：一种内存结构包含与指令相关的控制信号。
  - 控制存储。

今天的内容

- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计

单周期微架构

- 这是一个好的想法/设计吗？
- 什么时候这是一个好的设计？
- 什么时候这是一个不好的设计？
- 我们如何设计一个更好的微架构？

单周期微架构：时间由谁决定？

本节约定：每条指令从一个时钟边沿开始，到下一个边沿完成。

每条指令占1个周期

CPI=1。取指、读寄存器、运算等步骤在同一个周期内传播。

周期必须容纳最长路径

时钟周期至少覆盖最慢指令所需的组合传播与状态更新时间。

较快指令也使用同一周期

即使一条指令较早得到结果，也要等到统一的时钟更新边沿。

CPI=1 不等于每条指令所需的电路传播时间都相同。

最慢的指令是什么？

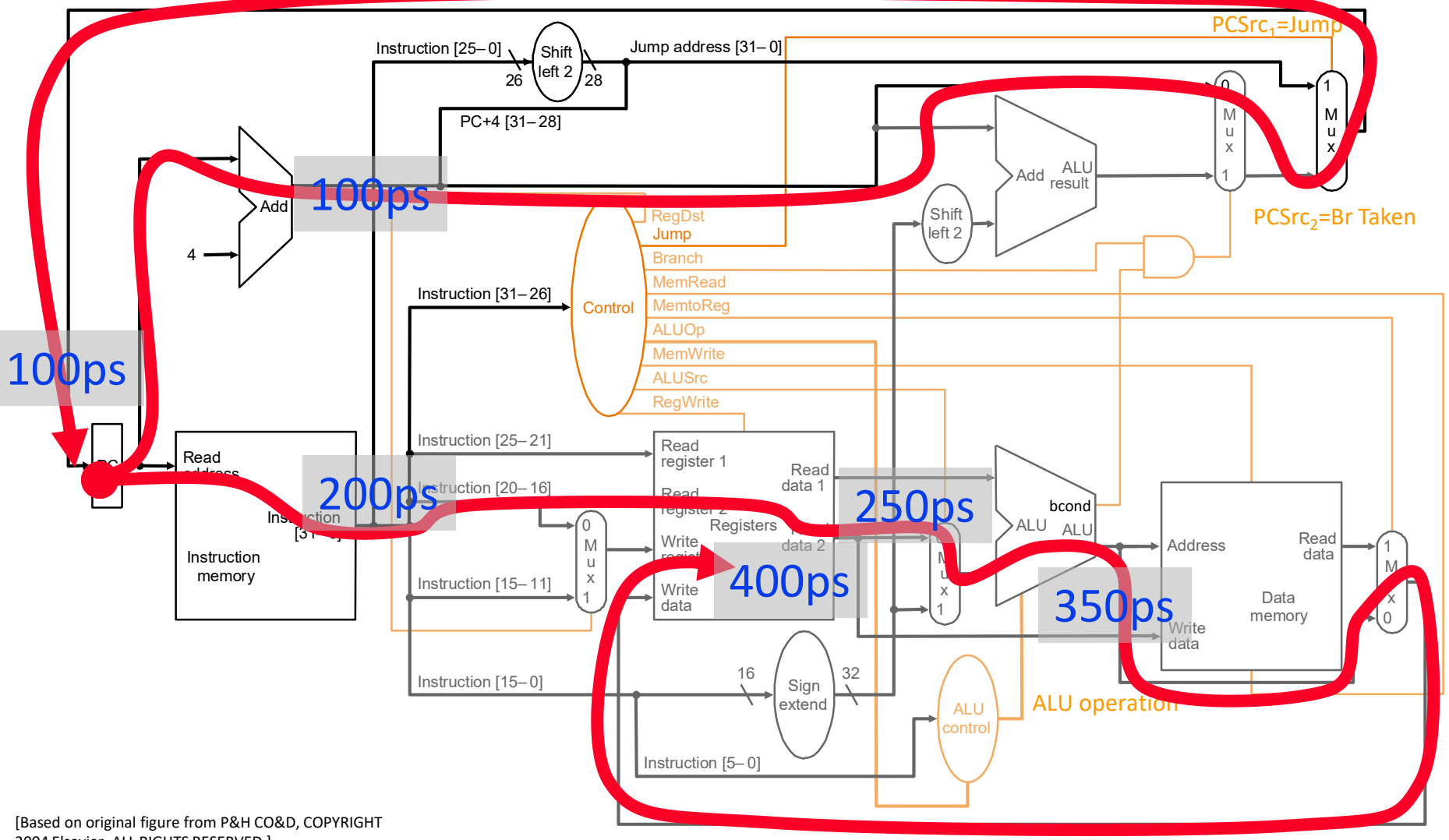
- 让我们回到基础
- 指令处理周期的5个阶段每个都需要一个机器时钟周期来完成
- 每个阶段是否对所有指令都需要相同的时间（延迟）？

单周期数据通路分析

- 假设：
  - 内存单元（读或写）：200 ps
  - ALU和加法器：100 ps
  - 寄存器文件（读或写）：50 ps
  - 其他组合逻辑：0 ps

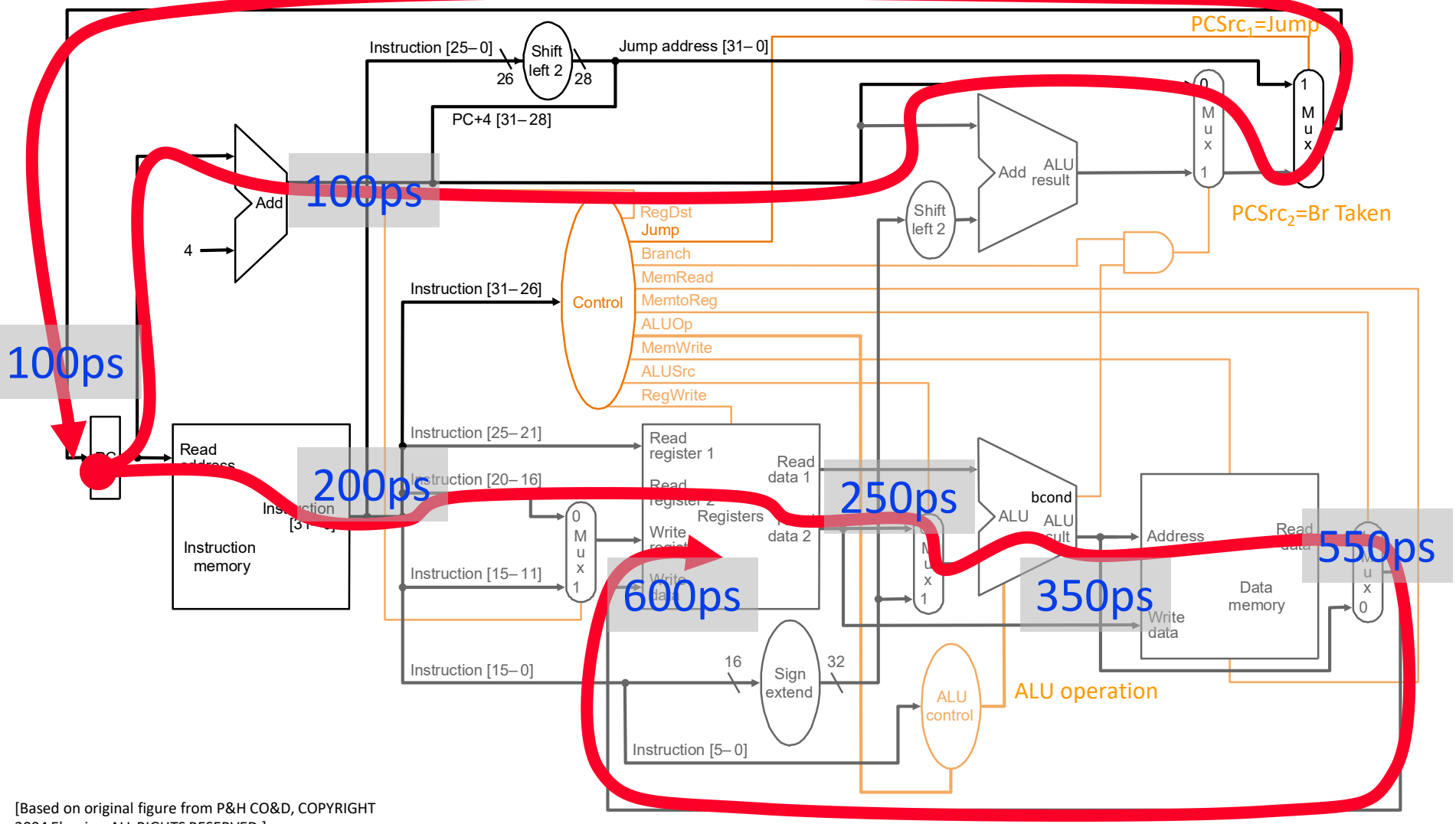
steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

R型和 I型 ALU 指令



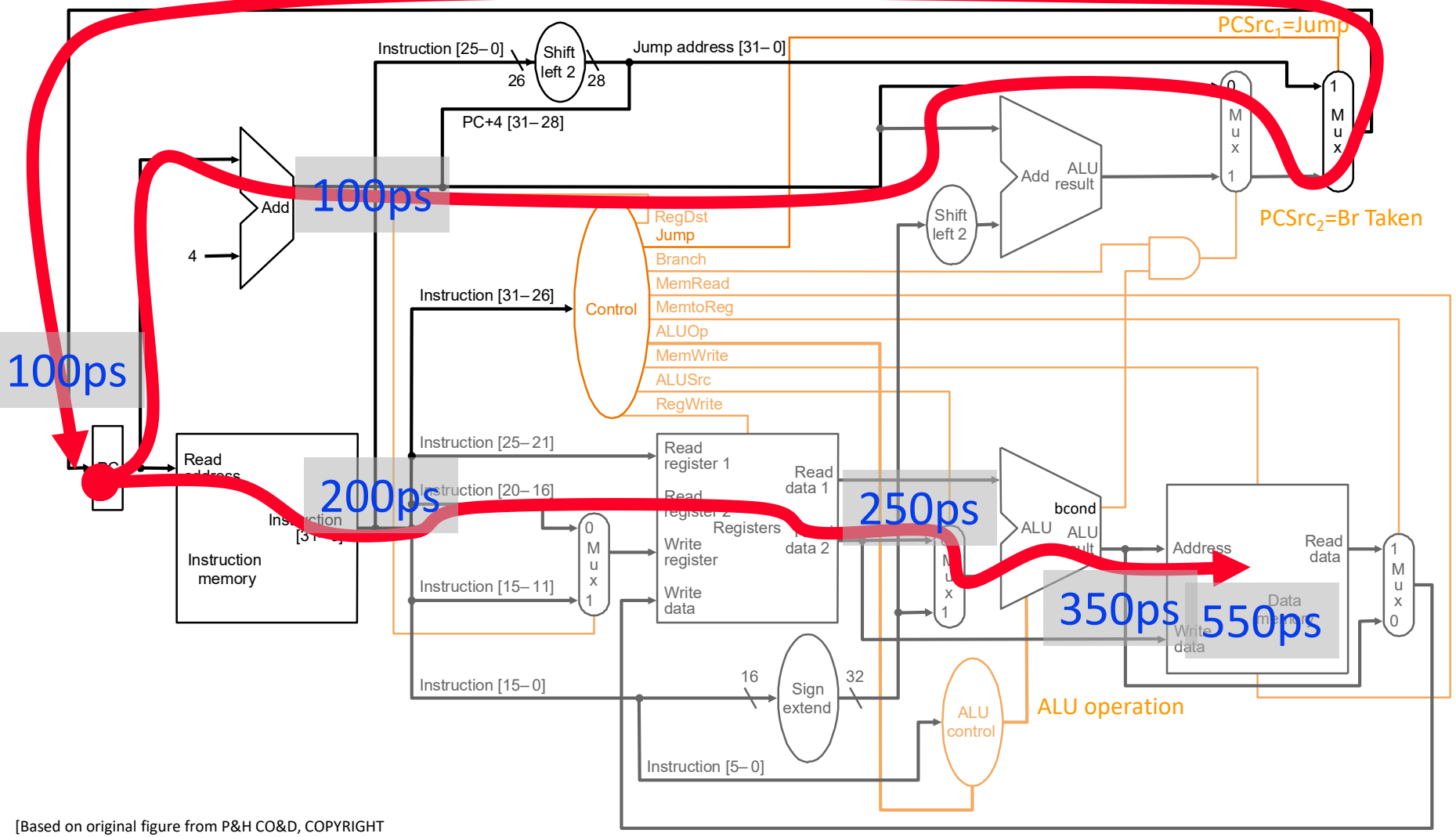
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

LW



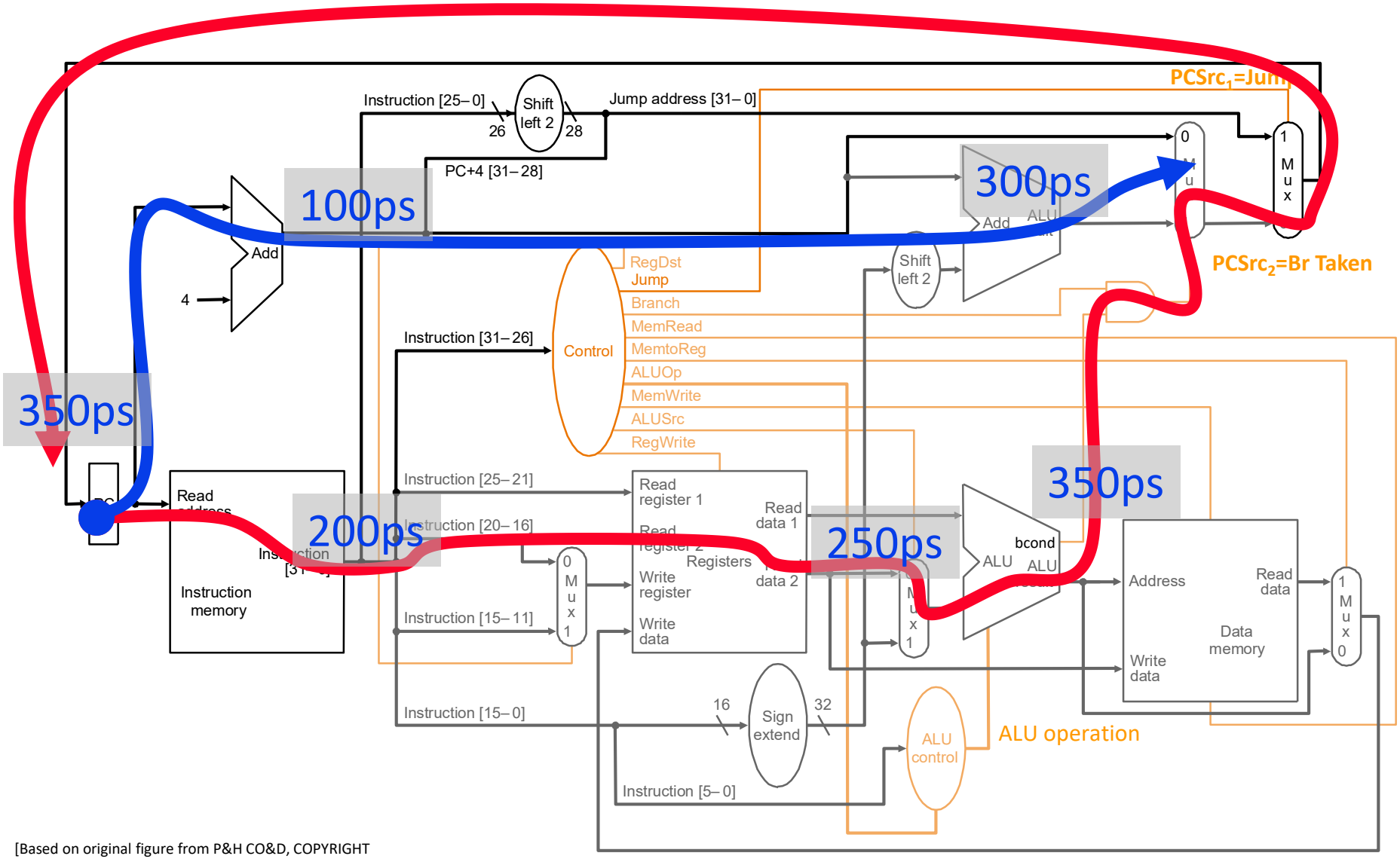
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

SW



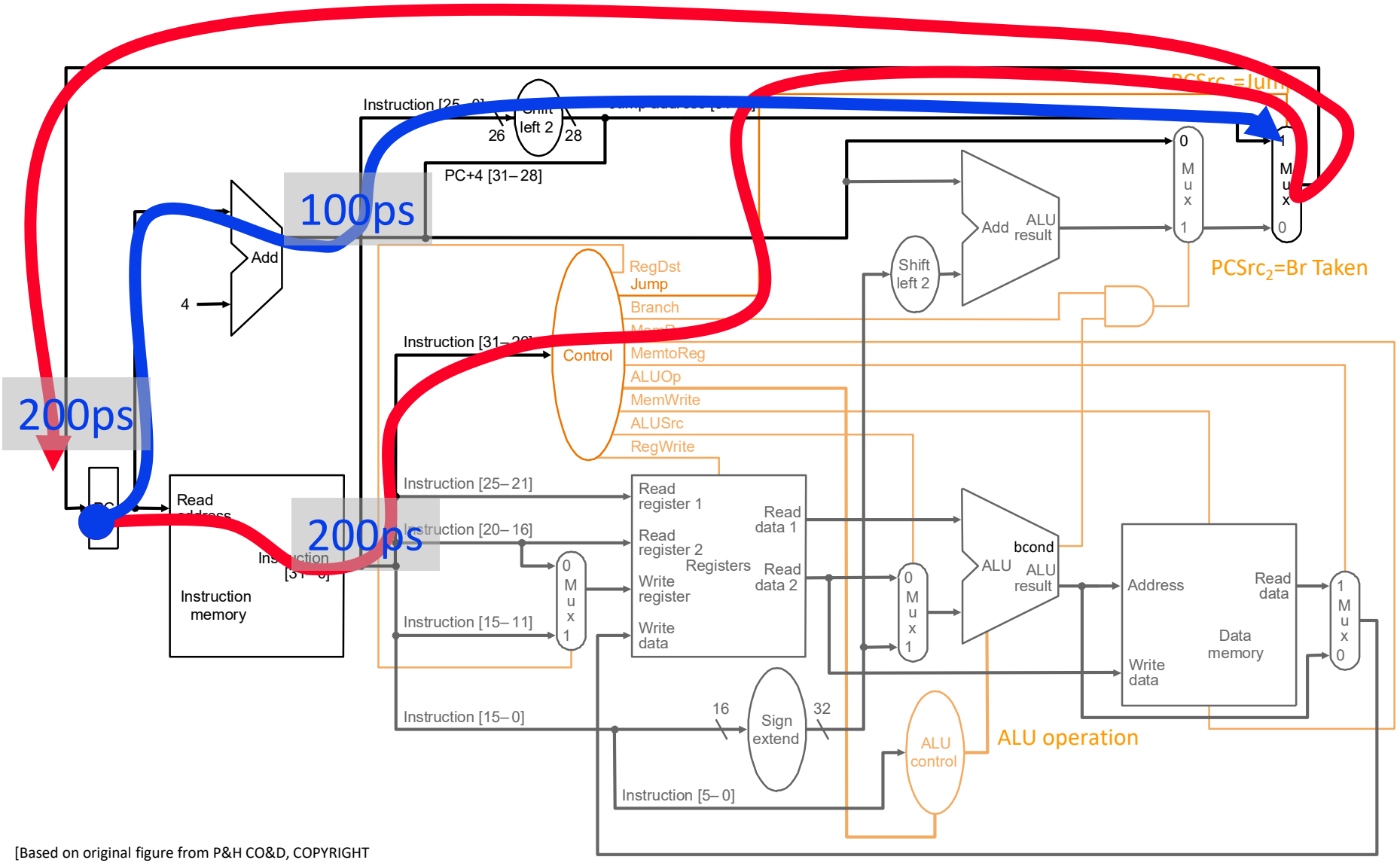
[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

条件分支（跳转）



[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

跳转



[Based on original figure from P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

控制逻辑如何影响？

- 这如何影响关键路径？
- 思考：
  - 控制逻辑能否出现在关键路径上？
  - 关于CDC 5600：控制存储器访问时间过长...

最慢的指令处理是什么？

- 内存不是魔法。
- 如果内存有时需要100ms来访问呢？
- 让简单的寄存器对寄存器加法或跳转操作耗时{100ms + 其他所有内存操作}，有意义吗？
- 如果你需要多次访问内存才能处理一条指令怎么办？
  - 哪些指令需要这样做？
  - 是否为内存提供多个端口？

单周期微架构：复杂性

- 低效
  - 所有指令执行的速度与最慢指令相同。
  - 必须并行提供最坏情况组合资源，以满足任何指令的需求。
  - 如果某个资源在指令处理周期的不同部分多次使用，则需要复制该资源。
- 不一定是实现ISA的最简单方式
  - 单周期实现REP MOVS（x86）或INDEX（VAX）？
- 不容易优化/提高性能
  - 优化常见情况无法解决问题（例如，常见指令）。
  - 需要始终优化最坏情况。

（微）架构设计原则

- **关键路径设计**
  - 查找并减少最大组合逻辑延迟
  - 如果延迟过长，将路径分成多个周期
- **基础（常见情况）设计**
  - 在最重要的地方花费时间和资源
    - 即，改善机器真正设计要做的事情
  - 常见情况与不常见情况
- **平衡设计**
  - 平衡指令/数据流通过硬件组件
  - 设计以消除瓶颈：为工作平衡硬件

单周期设计 vs. 设计原则

- 关键路径设计
- 基础（常见情况）设计
- 平衡设计
- 单周期微架构如何在这些原则下表现？

今天的内容

- 微架构--单周期CPU设计
- 单周期CPU数据通路(Single-Cycle Datapath)
  - 算术和逻辑运算指令
  - 数据移动指令
  - 控制流指令
- 单周期CPU控制逻辑
- 评估微架构--单周期CPU
- 微架构—多周期CPU设计